

COMPUTACIÓN DIGITAL

NRC: 24870

INTEGRANTES:

LENIN ANTONIO BARRIONUEVO RODRIGUEZ

MARCO ADRIAN PADILLA TRIVIÑO

ANGELO RAPHAEL GALARZA MANOSALVAS

FECHA: 10 de agosto de 2025

Arduino Semáforo Inteligente

COMPUTACIÓN DIGITAL.

ING. MIROSLAVA ARACELY ZAPATA RODRIGUEZ

Departamento de Eléctrica, Electrónica y Telecomunicaciones- ESPE



10 de agosto de 2025

Índice

1. Introducción	6
2. Objetivos	6
2.1. Objetivo General	6
2.2. Objetivos Específicos	7
3. Materiales y Equipos Utilizados	7
3.1. Hardware Principal	7
3.2. Componentes de Visualización	7
3.3. Componentes de Entrada	8
3.4. Componentes de Audio	8
3.5. Componentes Electrónicos Auxiliares	8
3.6. Software y Herramientas de Desarrollo	9
3.7. Herramientas de Programación de Memoria	9
3.8. Software de Comunicación Serial	9
3.9. Herramientas de Medición y Prueba	9
3.10. Librerías de Software Utilizadas	10
3.11. Materiales de Soporte	10
4. Marco Teórico	10
5. Justificación	15
6. Metodología y Desarrollo	16
6.1. Codificación de la EEPROM AT28C64B para Visualización de Letras	16
6.2. Proceso de programación de la memoria	17



6.3. Armado en Protoboard	17
6.4. Programación de Arduino	18
6.5. Desarrollo de la interfaz en Visual Studio (C#)	21
6.6. Comunicación Serial (Hardware Físico)	22
6.7. Simulación del sistema completo en Proteus	23
6.8. Comunicación Serial (Simulación Virtual)	24
6.9. Almacenamiento en base de datos SQLite	25
6.10. Visualización de gráficas estadísticas	27
6.11. Interacción total y validación	27
7. Configuración de Pines y Componentes del Sistema	31
7.1. Distribución Completa de Pines del Arduino UNO	31
7.2. Diagrama Visual de Conexiones del Sistema	31
7.3. Tabla Detallada de Conexiones por Componente	33
7.3.1. Detalles Técnicos de las Conexiones	34
7.3.2. Consideraciones de Diseño del Cableado	34
7.4. Diagrama ASCII del Sistema Completo	35
7.5. Agrupación por Módulos Funcionales	36
7.5.1. Sistema de Semáforo LED	36
7.5.2. Display de 7 Segmentos con EEPROM AT28C64B	37
7.6. Sistema de Entrada con Teclado Matricial 4x4	37
7.6.1. Configuración del Teclado	37
7.6.2. Funciones Asignadas a cada Tecla	37
7.6.3. Implementación en el Código C#	38
7.6.4. Matriz de LEDs 8x8 (MAX7219)	39
7.7. Sistema de Audio Avanzado con Buzzer	39
7.7.1. Melodías Programadas por Estado	39
7.7.2. Implementación de Melodías	40
7.7.3. Integración con Estados del Sistema	40
7.8. Display LCD 16x2 con Interfaz I2C	41
7.8.1. Características del Módulo LCD I2C	41
7.8.2. Información Mostrada en el LCD	41
7.8.3. Conexión I2C	41



7.9. Sistema de Matriz de LEDs 8x8 con MAX7219	42
7.9.1. Características del MAX7219	43
7.9.2. Patrones Implementados	43
7.9.3. Animaciones Implementadas	43
7.9.4. Código de Control de la Matriz	44
8. Implementación de Funcionalidades Avanzadas	45
8.1. Sistema de Matriz de LEDs 8x8 con MAX7219	45
8.1.1. Características del MAX7219	45
8.1.2. Patrones Implementados	45
8.1.3. Animaciones Implementadas	46
8.2. Sistema de Audio con Buzzer	47
8.2.1. Melodías Programadas	47
8.3. Teclado de Control Simplificado	47
8.3.1. Funciones de las Teclas	47
9. Integración y Comunicación entre Módulos	48
9.1. Protocolo de Comunicación Serial	48
9.1.1. Formato de Mensajes	48
9.2. Sincronización Hardware-Software	48
10.Resultados y Validación del Sistema	49
10.1. Pruebas de Funcionalidad	49
10.1.1. Pruebas de Hardware	50
10.1.2. Pruebas de Software	51
10.1.3. Pruebas de Integración	51
10.2. Métricas de Rendimiento	51
10.3. Casos de Uso Validados	51
11.Sistema de Retroalimentación Sonora	52
11.1. Archivos de Sonido Implementados	52
11.2. Implementación de Reproducción de Audio	52
11.3. Integración con Hardware	53
12.Conclusiones	53



13.Aplicación a la vida real — Propuesta	54
14.Componentes para implementación a escala real	57



1. Introducción

El presente informe técnico documenta el desarrollo completo de un sistema de semáforo inteligente, integrando tecnologías de software y hardware para simular y controlar el comportamiento vial tanto de vehículos como de peatones. El proyecto se ha llevado a cabo en un entorno de ingeniería, combinando programación en C#, comunicación serial, diseño de interfaz gráfica en Windows Forms, integración con base de datos SQLite y simulación electrónica mediante Proteus y Arduino UNO.

La propuesta central del proyecto es establecer un entorno mixto (físico y virtual) que permita la interacción entre un sistema programado en un entorno de desarrollo visual, una tarjeta de prototipado físico (Arduino), una memoria EEPROM y una simulación paralela en Proteus, incluyendo la funcionalidad de pulsadores virtuales que simulan acciones del hardware real.

El sistema no solo permite el control visual del flujo vehicular y peatonal, sino que también registra, analiza y representa gráficamente los eventos ocurridos durante su ejecución, ofreciendo estadísticas visuales y almacenamiento persistente de datos. De esta forma, se evidencia una clara vinculación entre conceptos fundamentales de computación digital, electrónica, arquitectura Von Neumann, procesamiento de señales, gestión de datos y visualización dinámica de información.

Esta integración busca fortalecer el conocimiento técnico aplicado, promoviendo habilidades multidisciplinarias que abarcan desde el modelado lógico hasta la implementación de interfaces hombre-máquina funcionales, dentro de un marco académico riguroso y técnicamente sólido.

2. Objetivos

2.1. Objetivo General

Diseñar e implementar un sistema integrado de semáforo inteligente que permita la interacción entre una interfaz gráfica programada en Visual Studio, un circuito físico con Arduino UNO y EEPROM AT28C64B, y una simulación virtual en Proteus, logrando la visualización, control, registro y análisis estadístico del comportamiento vial de vehículos y cruce peatonal.



2.2. Objetivos Específicos

- Desarrollar una aplicación de escritorio en C# que simule el funcionamiento de un semáforo, registre eventos y permita visualizar estadísticas gráficas.
- Integrar comunicación serial entre el entorno de Visual Studio y el microcontrolador Arduino UNO para el control de periféricos físicos (luces y pulsadores).
- Configurar y utilizar una memoria EEPROM externa (AT28C64B) como parte del circuito físico, con funcionalidades reales de almacenamiento y lectura de datos.
- Diseñar y simular el mismo sistema en Proteus para reemplazar temporalmente al prototipo físico y validar el comportamiento desde el entorno virtual.
- Implementar una base de datos SQLite para almacenar eventos generados durante la simulación del sistema y generar gráficos estadísticos desde dichos registros.
- Permitir el control del sistema desde el circuito simulado en Proteus mediante un pulsador virtual, manteniendo la funcionalidad sin desconectar el resto del sistema.
- Evaluar el comportamiento del sistema a través de gráficos y estadísticas visuales que representen los tipos de eventos y características de los vehículos registrados.

3. Materiales y Equipos Utilizados

3.1. Hardware Principal

- **Arduino UNO R3** - Microcontrolador principal basado en ATmega328P
- **Memoria EEPROM AT28C64B** - Memoria no volátil de 8KB (8192 x 8 bits)
- **Protoboard de 830 puntos** - Base para montaje del circuito
- **Cable USB tipo A-B** - Comunicación y alimentación del Arduino

3.2. Componentes de Visualización

- **Display de 7 segmentos cátodo común** - Visualización de letras según color de vehículo



- **Matriz de LEDs 8x8 con MAX7219** - Visualización gráfica avanzada y animaciones
- **LEDs de 5mm:**
 - 1x LED rojo (semáforo vehicular)
 - 1x LED amarillo (semáforo vehicular)
 - 1x LED verde (semáforo vehicular)
 - 1x LED verde adicional (paso peatonal)

Display LCD 16x2 con interfaz I2C - Visualización de mensajes del sistema y estado actual

3.3. Componentes de Entrada

- **Teclado matricial 4x4 completo** - Control avanzado del sistema con 16 teclas
- **Pulsador normalmente abierto** - Control manual del sistema físico

3.4. Componentes de Audio

- **Buzzer pasivo de 5V** - Generación de melodías según estado del semáforo

3.5. Componentes Electrónicos Auxiliares

- **Resistencias de carbón 1/4W:**
 - 4x Resistencias de 220 (protección de LEDs)
 - 7x Resistencias de 330 (display 7 segmentos)
 - 1x Resistencia de 10k (pull-up del pulsador)
- **Cables jumper macho-macho** - Conexiones en protoboard (pack de 40 unidades)
- **Cables jumper macho-hembra** - Conexiones Arduino-protoboard (pack de 20 unidades)



3.6. Software y Herramientas de Desarrollo

- **Arduino IDE 2.x** - Entorno de desarrollo para programación del microcontrolador
- **Microsoft Visual Studio 2022** - Desarrollo de la interfaz gráfica en C#
- **.NET Framework 4.8** - Framework para desarrollo de aplicaciones Windows
- **Proteus 8 Professional** - Simulación electrónica y virtual del circuito
- **SQLite** - Sistema de base de datos local para almacenamiento de eventos
- **DB Browser for SQLite** - Herramienta de administración de base de datos

3.7. Herramientas de Programación de Memoria

- **Programador universal de memorias EEPROM** - Para grabar la tabla de verdad en AT28C64B
- **Software Max Loader** - Interfaz para programación de memorias EEPROM
- **Archivo binario (.bin)** - Tabla de verdad codificada para los 8 estados del display

3.8. Software de Comunicación Serial

- **HyperTerminal** - Monitoreo y depuración de comunicación serial
- **Virtual Serial Port Driver (com0com)** - Creación de puertos COM virtuales
- **Módulo COMPIM de Proteus** - Interfaz de comunicación serial en simulación

3.9. Herramientas de Medición y Prueba

- **Multímetro digital** - Verificación de voltajes y continuidad
- **Osciloscopio virtual (Proteus)** - Análisis de señales digitales
- **Monitor serial del Arduino IDE** - Depuración de comunicación



3.10. Librerías de Software Utilizadas

- **System.IO.Ports** - Comunicación serial en C#
- **System.Data.SQLite** - Manejo de base de datos SQLite
- **System.Windows.Forms.DataVisualization.Charting** - Generación de gráficos
- **LedControl.h** - Control de matriz MAX7219 en Arduino
- **SoftwareSerial.h** - Comunicación serial adicional en Arduino

3.11. Materiales de Soporte

- **Fuente de alimentación externa 9V** - Alimentación independiente del circuito
- **Jack de alimentación 2.1mm** - Conector para fuente externa
- **Capacitor electrolítico 1000F/16V** - Filtrado de alimentación
- **Cinta aislante** - Aislamiento de conexiones
- **Tornillos y soportes** - Montaje mecánico del prototipo

4. Marco Teórico

El desarrollo de un sistema de semáforo inteligente que integra hardware, software y simulación requiere el dominio de varios conceptos y herramientas tecnológicas. En esta sección se abordan los fundamentos teóricos relacionados con los componentes utilizados, tanto físicos como virtuales, y se justifica su aplicación en el contexto del proyecto.

Arquitectura de Comunicacion Serial

La comunicación serial es un método de transmisión de datos en el que los bits se envían uno a uno a través de un único canal. Este tipo de comunicación es ampliamente utilizado en sistemas embebidos debido a su simplicidad y bajo costo en conexión. En este proyecto se aprovecha el protocolo UART (Universal Asynchronous Receiver Transmitter) para establecer la comunicación entre la computadora (interfaz grafica), el Arduino UNO físico y, posteriormente, el simulador Proteus.

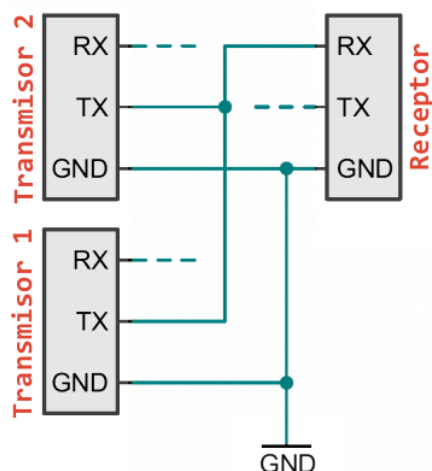


Figura 1: Diagrama de una arquitectura serial.

Microcontrolador Arduino UNO

El Arduino UNO es una placa de desarrollo basada en el microcontrolador ATmega328P. Cuenta con 14 pines digitales, 6 entradas analógicas, comunicación serial, I2C y SPI, además de una interfaz USB para programación y comunicación con la PC. Este microcontrolador fue utilizado como el corazón del sistema de control, encargado de gestionar los estados del semáforo y almacenar los eventos ocurridos.

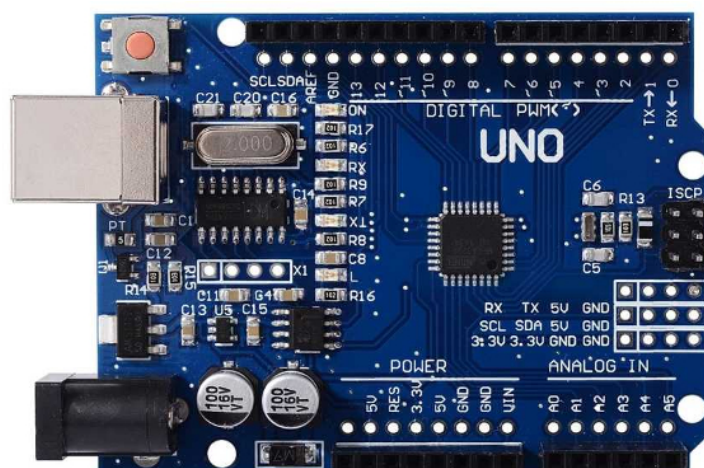


Figura 2: Microcontrolador Arduino UNO.

Memoria EEPROM AT28C64B

La AT28C64B es una memoria EEPROM (Electrically Erasable Programmable Read-Only Memory) paralela de 8KB (8192 x 8 bits). Esta memoria se utiliza en el proyecto para almacenar datos de forma no volátil, permitiendo que los eventos y configuraciones persistan incluso después de reinicios. Se comunica con el microcontrolador mediante un bus paralelo, y requiere un adecuado direccionamiento de pines para su escritura y lectura.

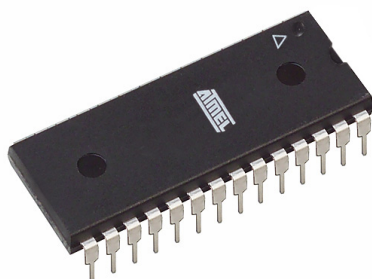


Figura 3: Circuito integrado AT28C64B.

28C64 Pinout

RDY/BUSY (or NC)	1	28	VCC
A12	2	27	WE
A7	3	26	NC
A6	4	25	A8
A5	5	24	A9
A4	6	23	A11
A3	7	22	OE
A2	8	21	A10
A1	9	20	CE
A0	10	19	I/O7
I/O0	11	18	I/O6
I/O1	12	17	I/O5
I/O2	13	16	I/O4
GND	14	15	I/O3

Figura 4: Datasheet de circuito integrado AT28C64B.



Plataforma Visual Studio (.NET, WinForms)

Se utilizó Visual Studio para desarrollar una interfaz gráfica en C# utilizando Windows Forms (WinForms). Esta interfaz permite visualizar en tiempo real el estado del sistema, registrar eventos, generar estadísticas, graficar los resultados obtenidos desde la base de datos SQLite y gestionar la comunicación serial con el Arduino o con Proteus.

Base de Datos SQLite

SQLite es una base de datos relacional liviana, ideal para aplicaciones embebidas. En este proyecto se usó para registrar los eventos generados por el sistema: pasos de vehículos, colores, velocidades, tiempos de cruce, entre otros. La base de datos es creada y gestionada localmente desde el programa en C#, y se accede a ella mediante comandos SQL.

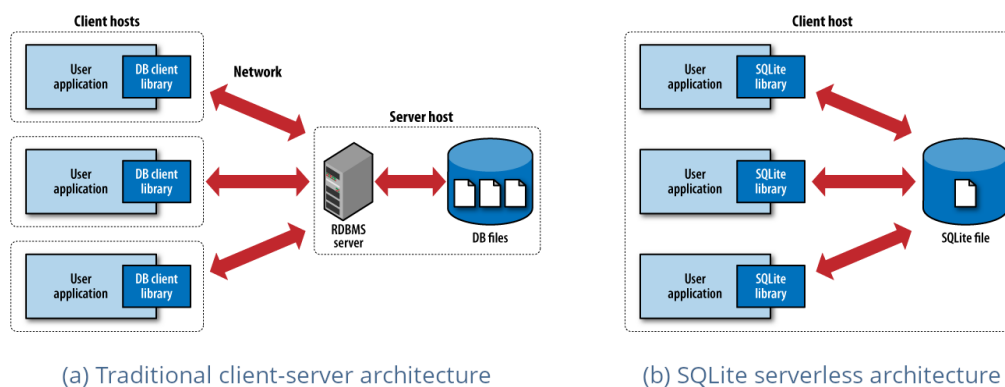


Figura 5: Arquitectura SQLite - BDD.

Software de Simulación Proteus 8

Proteus es un entorno de simulación electrónica que permite emular circuitos y microcontroladores. En este proyecto se utilizó para simular el funcionamiento del Arduino, el botón de entrada y el semáforo, logrando una representación digital del circuito físico. Se conectó Proteus al entorno Visual Studio a través de puertos seriales virtuales, usando COM5, COM1 y COM2 para simular la comunicación UART.

Puertos Seriales Virtuales (Virtual Serial Port)

Para facilitar la comunicación entre Proteus y el programa en Visual Studio sin necesidad de hardware físico, se utilizaron herramientas como Virtual Serial Port. Esta permite crear pares de puertos COM virtuales conectados entre sí (por ejemplo, $(COM1 < - > COM2)$), de forma que un extremo pueda ser utilizado por Proteus y el otro por el programa en C#. Asimismo, para la comunicación entre el programa en Visual con la parte física que sería el protoboard conectado con el Arduino UNO, utilizando el $(COM5)$.



Figura 6: Comunicación entre aplicaciones por Virtual Serial Port.

HyperTerminal y Monitoreo Serial

HyperTerminal es un software que permite enviar y recibir datos a través de puertos seriales. Fue utilizado para realizar pruebas de comunicación entre dispositivos y verificar el envío correcto de comandos desde la interfaz hacia el sistema embebido. Facilita el diagnóstico de errores y la observación de tráfico serial durante el desarrollo.

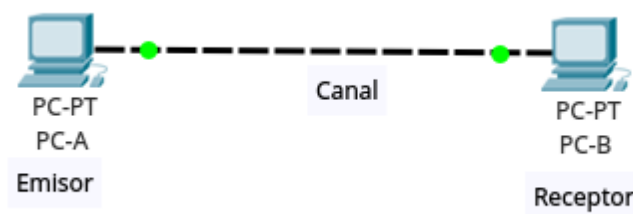


Figura 7: Uso de Hyperterminal.

Visualización de Datos con System.Windows.Forms.DataVisualization.Charting

La representación gráfica de los eventos almacenados en la base de datos se realizó mediante el componente **Chart** de WinForms. Se generaron gráficas de barras y pasteles que representan la cantidad de vehículos por color y la distribución de tipos de eventos registrados. Estas gráficas hacen uso de consultas SQL para obtener los datos desde SQLite y se actualizan automáticamente con cada sesión del sistema.



Figura 8: Tipos de gráficas realizadas con **Chart**.

5. Justificación

El desarrollo de un sistema de semáforo inteligente controlado por una interfaz en Visual Studio y sincronizado con un circuito físico y simulado, responde a la necesidad de integrar distintos entornos de desarrollo en proyectos de ingeniería aplicados. Esta implementación tiene un alto valor formativo y técnico, ya que permite la comprensión de cómo convergen el diseño de hardware, la programación embebida, el almacenamiento digital, la comunicación serial y el desarrollo de interfaces gráficas.

La propuesta combina componentes reales (Arduino UNO, memoria EEPROM AT28C64B, LEDs, protoboard) con entornos virtuales de simulación (Proteus 8, puertos COM virtuales, COMPIM), permitiendo validar el comportamiento de un sistema híbrido desde múltiples frentes. Esta integración es especialmente útil en contextos educativos, de prototipado rápido o en entornos donde se requiera emular el comportamiento de sistemas físicos sin depender de todos los recursos materiales.

Además, se incorporan herramientas modernas como bases de datos SQLite y gráficos



estadísticos generados dinámicamente desde la aplicación en C#, lo que aporta una capa adicional de análisis y monitoreo. Esto es crucial en sistemas embebidos de control y registro, ya que no solo importa el comportamiento del sistema, sino también su trazabilidad y capacidad de generar reportes interpretables.

La justificación técnica también se fundamenta en el uso didáctico del proyecto: se puede demostrar visualmente cómo un autómata de control basado en EEPROM puede ser programado para definir una lógica de transición de estados, reemplazando estructuras de software convencionales. Así mismo, se muestra cómo interactúan múltiples niveles del sistema: bajo nivel (hardware), medio nivel (firmware y comunicación serial), y alto nivel (aplicación de escritorio con análisis de datos).

Finalmente, esta implementación promueve el aprendizaje integral en sistemas embebidos, programación, electrónica digital y simulación, con un enfoque práctico que puede ser aplicado en semáforos reales, sistemas de control de tráfico, automatización de procesos y entornos de prueba para estudiantes e ingenieros en formación.

6. Metodología y Desarrollo

Esta sección describe paso a paso la construcción del sistema completo, integrando hardware, software y simulación. Desde el diseño de la lógica de control utilizando una EEPROM como máquina de estados, hasta la visualización de eventos y estadísticas en una interfaz desarrollada en C#.

6.1. Codificación de la EEPROM AT28C64B para Visualización de Letras

Una parte fundamental del sistema de semáforo inteligente es la forma en que se visualiza, mediante un **display de 7 segmentos**, la letra correspondiente al color del vehículo que está cruzando el paso cebra. Esta funcionalidad fue implementada utilizando una memoria EEPROM paralela modelo **AT28C64B**, que contiene una **tabla de verdad codificada** con los patrones de salida adecuados para cada letra que debe mostrar el display.

La memoria AT28C64B fue programada con una tabla de verdad que asocia combinaciones de entrada A2, A1, A0 (usadas como selección de dirección de memoria) con un byte de salida (8 bits) que representa el patrón de encendido de los segmentos del display para mostrar una letra específica. La codificación fue la siguiente:



Entradas EEPROM			Pines del Display 7 Segmentos								Resultado	Visualización
Direcciones			Segmentos (MSB a LSB)								Codificado	Mostrada
A2	A1	A0	h	g	f	e	d	c	b	a	Salida (Hex)	Letra / Color
0	0	0	0	1	0	1	0	0	0	0	50	R (rojo)
0	0	1	0	1	1	1	0	1	1	1	77	A (azul)
0	1	0	0	0	0	1	1	0	0	0	1C	V (verde)
0	1	1	0	1	1	1	0	1	1	1	77	A (amarillo)
1	0	0	0	1	0	1	0	1	0	0	54	N (negro)
1	0	1	0	1	1	1	1	1	1	1	7F	B (blanco)
1	1	0	0	1	0	1	0	1	0	0	54	N (naranja)
1	1	1	0	0	1	1	1	0	0	0	38	L (lila)

Cuadro 1: Tabla de verdad codificada para representar letras según color del vehículo en display de 7 segmentos.

6.2. Proceso de programación de la memoria

La EEPROM AT28C64B fue cargada con estos valores utilizando el software **Max Loader**, un programa que permite leer, escribir y verificar contenidos en memorias EEPROM a través de un programador externo. El procedimiento seguido fue el siguiente:

1. Se generó un archivo binario con los 8 bytes de salida correspondientes a cada combinación de entrada (de 000 a 111).
2. Se abrió el programa Max Loader y se cargó el archivo binario.
3. Se seleccionó el modelo de memoria AT28C64B dentro del software.
4. Se conectó la memoria al programador y se escribió el contenido.
5. Se realizó una verificación byte a byte para asegurar que los datos se grabaron correctamente.

6.3. Armado en Protoboard

El sistema físico se construyó utilizando una memoria EEPROM AT28C64B y un microcontrolador Arduino UNO. La EEPROM fue programada previamente con una tabla de

verdad que contiene la lógica de funcionamiento del semáforo. Cada dirección de memoria representa un estado, y los datos almacenados determinan las salidas de control.

- Se conectaron las direcciones A0 a A12 de la EEPROM a pines digitales del Arduino UNO configurados como salidas.
- Las salidas Q0 a Q7 de la memoria se utilizaron para controlar los LEDs del semáforo (rojo, amarillo, verde) y un display de 7 segmentos, el cual muestra la inicial del color del vehículo (por ejemplo, 'R' para rojo, 'V' para verde, 'A' para amarillo, etc.).
- El pin de **Output Enable** (OE) se mantuvo en bajo (activo) y el pin **Write Enable** (WE) se mantuvo en alto para garantizar sólo lectura.
- Se utilizaron resistencias de protección para los LEDs y display.

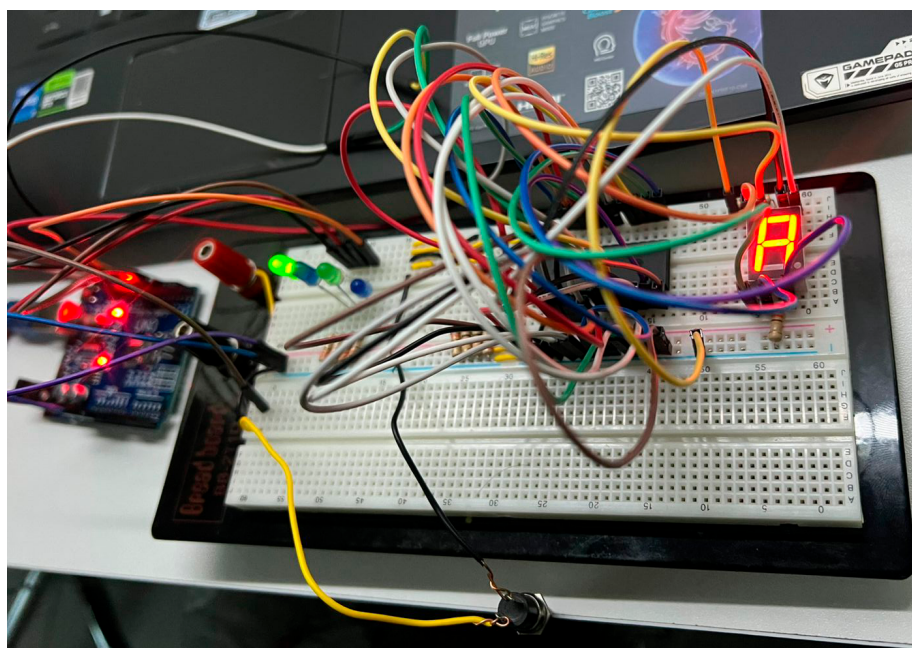


Figura 9: Montaje del protoboard con EEPROM, Arduino y display 7 segmentos.

6.4. Programación de Arduino

El Arduino se encargó de gestionar el control integral del semáforo inteligente mediante una arquitectura no bloqueante que coordina múltiples subsistemas. La programación im-



plementó una máquina de estados robusta para el manejo de transiciones y comunicación serial.

- Se implementó un sistema de comunicación serial no bloqueante que procesa comandos desde la aplicación externa C para cambiar estados del semáforo.
- La gestión de la EEPROM AT28C64B utiliza líneas de dirección controladas por Arduino para decodificar colores de vehículos y mostrarlos en el display de 7 segmentos.
- Se desarrolló un sistema multimedia que incluye animaciones en matriz LED 8x8 y melodías sincronizadas para cada estado del semáforo.
- La optimización de memoria mediante PROGMEM permite almacenar datos multimedia en memoria Flash, liberando la limitada SRAM del Arduino Uno.
- El código fue compilado en Arduino IDE y exportado como archivo `.hex` para implementación en Proteus.

Listing 1: Gestión de EEPROM y control de estados del semáforo

```
// Configuración de líneas de dirección EEPROM
const int EEPROM_A0 = 8, EEPROM_A1 = 9, EEPROM_A2 = 10;
String colores[] = {"Rojo", "Azul", "Verde", "Amarillo",
"Negro", "Blanco", "Naranja", "Lila"};
void mostrarColorEnDisplay(String color) {
int direccion = -1;
for (int i = 0; i < totalColores; i++) {
if (colores[i] == color) {
direccion = i;
break;
}
}
if (direccion != -1) {
digitalWrite(EEPROM_A0, (direccion & B001) ? HIGH : LOW);
digitalWrite(EEPROM_A1, (direccion & B010) ? HIGH : LOW);
digitalWrite(EEPROM_A2, (direccion & B100) ? HIGH : LOW);
}
```



```
}  
  
void procesarComando(String comando) {  
    if (comando != "APAGAR") programaActivo = true;  
    if (comando == "VERDE") {  
        apagarTodos();  
        digitalWrite(LED_VERDE, HIGH);  
        digitalWrite(LED_PEATON, LOW);  
        estadoActual = "VERDE";  
    } else if (comando == "ROJO") {  
        apagarTodos();  
        digitalWrite(LED_ROJO, HIGH);  
        digitalWrite(LED_PEATON, HIGH);  
        estadoActual = "ROJO";  
        mostrarEnDisplay('_');  
    }  
    // Actualizacion de LCD y estados...  
}
```



```
semaforoProject | Arduino IDE 2.3.6
File Edit Sketch Tools Help
Select Board

semaforoProject.ino
1 /**
2  * @file semaforoProject.ino
3  * @author [Lenin Barrionuevo | Adrián Padilla | Raphael Galarza]
4  * @brief Sistema de control integral para un semáforo inteligente.
5  * @version 2.0 (Refactorizado)
6  * @date 2024-07-31
7  *
8  * @details
9  * Este código controla un sistema de semáforo multifuncional que integra:
10  * - Control de LEDs para semáforo vehicular y peatonal.
11  * - Gestión de un display de 7 segmentos mediante una EEPROM externa (AT28C64B).
12  * - Lectura de un teclado matricial para comandos locales.
13  * - Reproducción de melodías de estado a través de un buzzer.
14  * - Despliegue de animaciones en una matriz de LEDs 8x8 (MAX7219).
15  * - Interfaz de comunicación serial para control remoto desde una aplicación anfitriona (Host).
16  * El sistema está diseñado sobre una arquitectura no bloqueante para máxima capacidad de respuesta.
17  *
18  * =====
19  * ⚡ MAPA DE ASIGNACIÓN DE PINES (ARDUINO UNO/NANO) ⚡
20  * =====
21  * PIN ARDUINO | IDENTIFICADOR | MÓDULO / DESCRIPCIÓN
22  * -----
23  * Digital 2 | MATRIZ_DIN | (Data IN) para la matriz de LEDs MAX7219.
24  * Digital 3 | LED_ROJO | LED rojo del semáforo vehicular.
25  * Digital 4 | LED_AMARILLO | LED amarillo del semáforo vehicular.
26  * Digital 5 | LED_VERDE | LED verde del semáforo vehicular.
27  * Digital 6 | LED_PEATON | LED verde para el cruce peatonal.
28  * Digital 7 | MATRIZ_CS | (Chip Select) para la matriz de LEDs MAX7219.
29  * Digital 8 | EEPROM_A0 | Línea de dirección 0 (LSB) para la EEPROM AT28C64B.
30  * Digital 9 | EEPROM_A1 | Línea de dirección 1 para la EEPROM AT28C64B.
31  * Digital 10 | EEPROM_A2 | Línea de dirección 2 (MSB) para la EEPROM AT28C64B.
32  * Digital 11 | KEYPAD_COL_1 | Columna 1 del teclado matricial.
33  * Digital 12 | KEYPAD_COL_2 | Columna 2 del teclado matricial.
34  * Digital 13 | MATRIZ_CLK | (Clock) para la matriz de LEDs MAX7219.
35  * Analog A0 | KEYPAD_ROW_1 | Fila 1 del teclado matricial (configurado como 1x4).
36  * Analog A1 | PIN_BUZZER | Buzzer para la salida de audio.
37  * Analog A2 | KEYPAD_COL_3 | Columna 3 del teclado matricial.
38  * Analog A3 | KEYPAD_COL_4 | Columna 4 del teclado matricial.
39  * Analog A4 | I2C_SDA | (Serial Data) para comunicación I2C (usado por el LCD).
40  * Analog A5 | I2C_SCL | (Serial Clock) para comunicación I2C (usado por el LCD).
41  *
42  * -----
43  * Para más detalles sobre la conexión de la EEPROM al display de 7 segmentos,
44  * referirse al diagrama ASCII al final de este archivo.
45  * =====
```

Figura 10: Iniciando programación en Arduino IDE.

6.5. Desarrollo de la interfaz en Visual Studio (C#)

La interfaz fue desarrollada con Windows Forms. Se creó un formulario principal que incluye:

- Un gráfico del semáforo animado.
- Un botón para simular el pulsador (modo software).
- Comunicación con el Arduino a través del puerto serial (COM).



- Registro en base de datos SQLite de cada evento.
- Un segundo formulario para mostrar gráficas estadísticas.

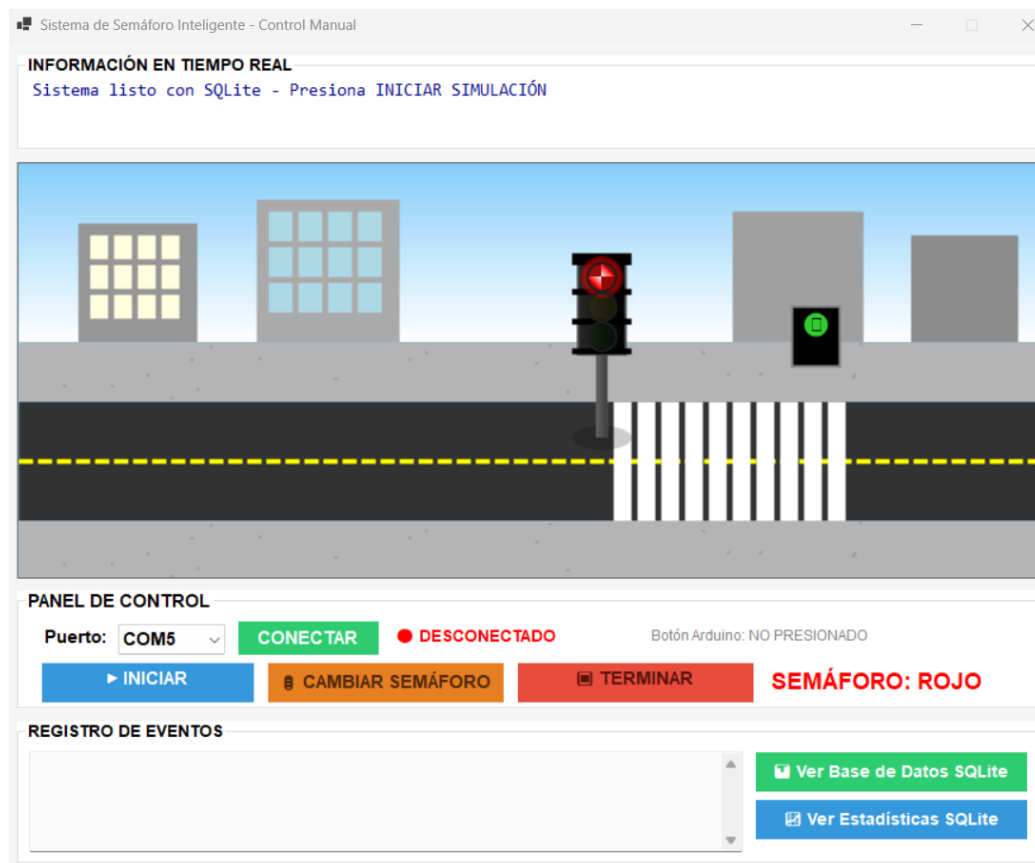


Figura 11: Montaje del protoboard con EEPROM, Arduino y display 7 segmentos.

6.6. Comunicación Serial (Hardware Físico)

La comunicación entre Arduino y el programa en C# se realiza mediante el puerto COM5, correspondiente al puerto USB del Arduino físico.

Listing 2: Inicialización del puerto en C#

```
SerialPort puerto = new SerialPort("COM5", 9600);  
puerto.DataReceived += Puerto_DataReceived;  
puerto.Open();
```



```
FormEstadisticas.cs  FormEstadisticas.Designer.cs  FormEstadisticas.cs [Design]  Form1.cs  Form1.resx  Program.cs  Form1.Designer.cs
SEMAFORO_AVANZADO  SEMAFORO_AVANZADO.Form1  Arduino
10  using System.Data.SQLite;
11  using System.Linq;
12  using System.IO;
13
14  namespace SEMAFORO_AVANZADO
15  {
16      3 references
17      public partial class Form1 : Form
18      {
19          // Puerto serial para Proteus
20          //SerialPort puerto = new SerialPort("COM2", 9600);
21
22          // Puerto serial para Arduino
23          private SerialPort arduino;
24
25          // Timer para animación
26          private System.Windows.Forms.Timer timerAnimacion;
27
28          // Base de datos SQLite - SÚPER FÁCIL
29          private string dbPath = "SemaforoDB.sqlite";
30          private string connectionString = "";
31
32          // Posiciones
33          private int posicionCarro = -80;
34          private int posicionPeatonY = 350;
35          private bool peatonCruzando = false;
36          private int peatonAnimacion = 0;
```

Figura 12: Inicialización del puerto COM5 para conectar a Arduino.

El Arduino envía datos como líneas de texto delimitadas por salto de línea, que son interpretadas por el software.

6.7. Simulación del sistema completo en Proteus

El circuito físico fue replicado virtualmente en Proteus 8, incluyendo:

- Arduino UNO con .hex cargado desde el Arduino IDE.
- EEPROM AT28C64B con .bin generado desde el programa físico.
- Pulsador simulado.
- LEDs y display de 7 segmentos.
- Módulo COMPIM para recibir/simular datos del puerto serial.

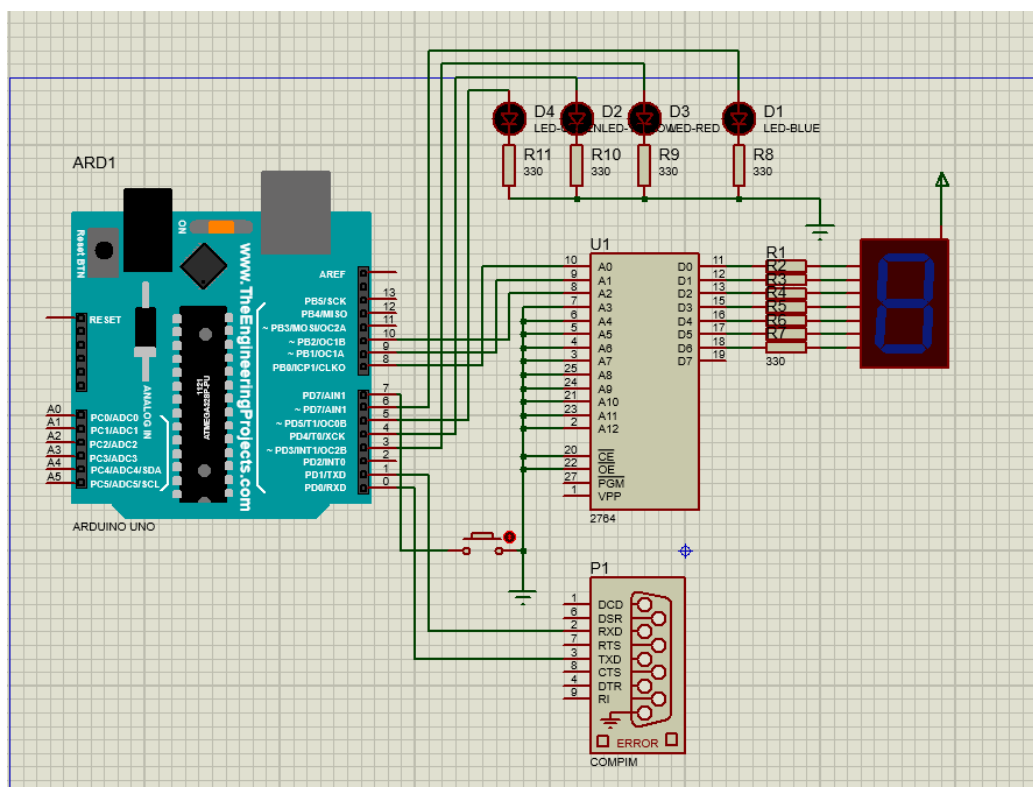


Figura 13: Simulación en Proteus con Arduino, memoria y pulsador conectado a COMPIM.

6.8. Comunicación Serial (Simulación Virtual)

Para que Visual Studio reciba eventos desde el botón de Proteus, se implementó:

- Un par virtual de puertos seriales (por ejemplo, COM1 y COM2) usando `com0com` o `Virtual Serial Port Driver`.
- COMPIM en Proteus configurado a COM1.
- Visual Studio configurado para recibir en COM2.
- Cuando se pulsa el botón en Proteus, se envía un carácter que es recibido por el programa como si fuera el botón físico.

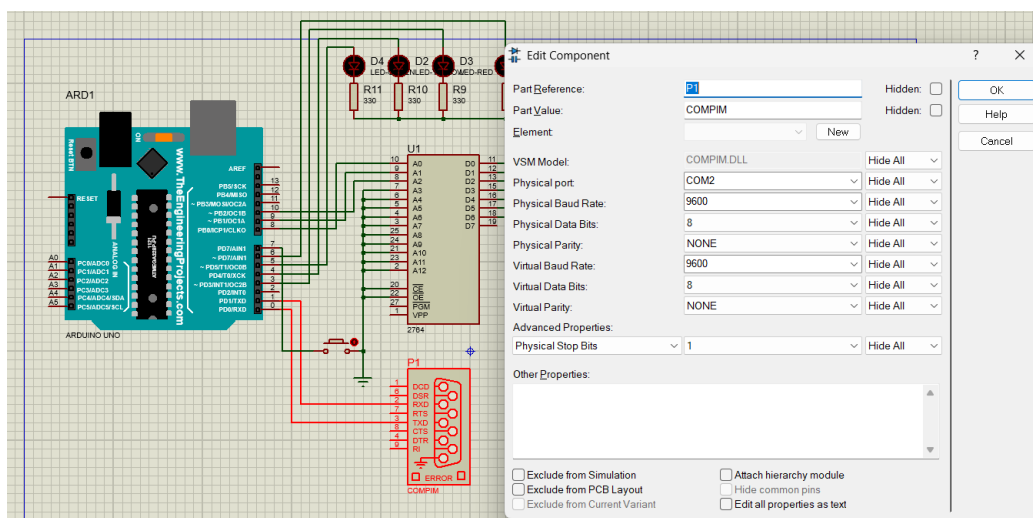


Figura 14: Configuración de COM2 en el COMPIM.

6.9. Almacenamiento en base de datos SQLite

Cada vez que se detecta un evento, el programa en C#:

1. Lee el evento enviado por Arduino.
2. Lo interpreta (tipo, color, tiempo).
3. Lo almacena en una base de datos SQLite local: `SemaforoDB.sqlite`.

Los datos que se almacenarían son:

ID	TipoEvento	ColorVehiculo	Tiempo	Descripcion
1	Carro	Verde	50.0	Carro completó recorrido
2	Peatón	—	12.3	Peatón cruzó exitosamente

Cuadro 2: Bosquejo de registros en la base de datos.



Figure 15 shows the DB Browser for SQLite interface displaying the database structure of the 'RegistroEventos' table. The table is defined with the following columns and data types:

Column Name	Data Type	Schema
ID	INTEGER	"ID" INTEGER PRIMARY KEY AUTOINCREMENT
FechaHora	DATETIME	"FechaHora" DATETIME NOT NULL
TipoEvento	TEXT	"TipoEvento" TEXT NOT NULL
ColorVehiculo	TEXT	"ColorVehiculo" TEXT
NumeroVehiculo	INTEGER	"NumeroVehiculo" INTEGER
EstadoSemaforo	TEXT	"EstadoSemaforo" TEXT NOT NULL
TiempoTranscurrido	REAL	"TiempoTranscurrido" REAL
Velocidad	INTEGER	"Velocidad" INTEGER
Observaciones	TEXT	"Observaciones" TEXT

Figura 15: Visualización de la estructura de la BDD en DB Browser.

Figure 16 shows the DB Browser for SQLite interface displaying the data in the 'RegistroEventos' table. The table contains the following records:

ID	FechaHora	TipoEvento	ColorVehiculo	NumeroVehiculo	EstadoSemaforo	TiempoTranscurrido	Velocidad	Observaciones
1	2025-07-04 10:54:16.2638302	Carro	Amarillo	1	VERDE	12.4136214	50	Carro completó recorrido
2	2025-07-04 10:54:27.8232593	Peatón		1	ROJO	23.9740328	0	Peatón cruzó exitosamente
3	2025-07-04 10:54:41.1056803	Resumen		1	ROJO	37.256462	0	Ciclos: 0, Peatones: 1, Colores: ...

Figura 16: Visualización de la BDD en DB Browser.

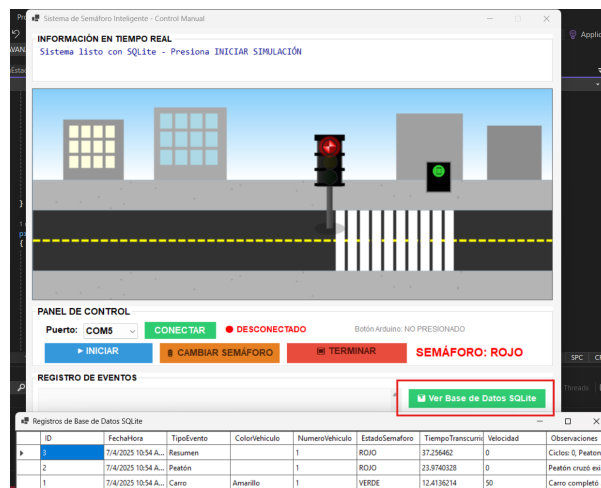


Figura 17: Visualización de la tabla principal de la BDD desde el programa.



6.10. Visualización de gráficas estadísticas

Se creó un formulario adicional (**FormEstadisticas**) que presenta dos gráficos generados dinámicamente desde la base de datos:

- **Gráfico de columnas:** Cantidad de vehículos por color.
- **Gráfico circular:** Tipos de evento registrados (carros, peatones).

Se utilizó el control **Chart** de **C#** con consultas **SQL** para contar agrupaciones.

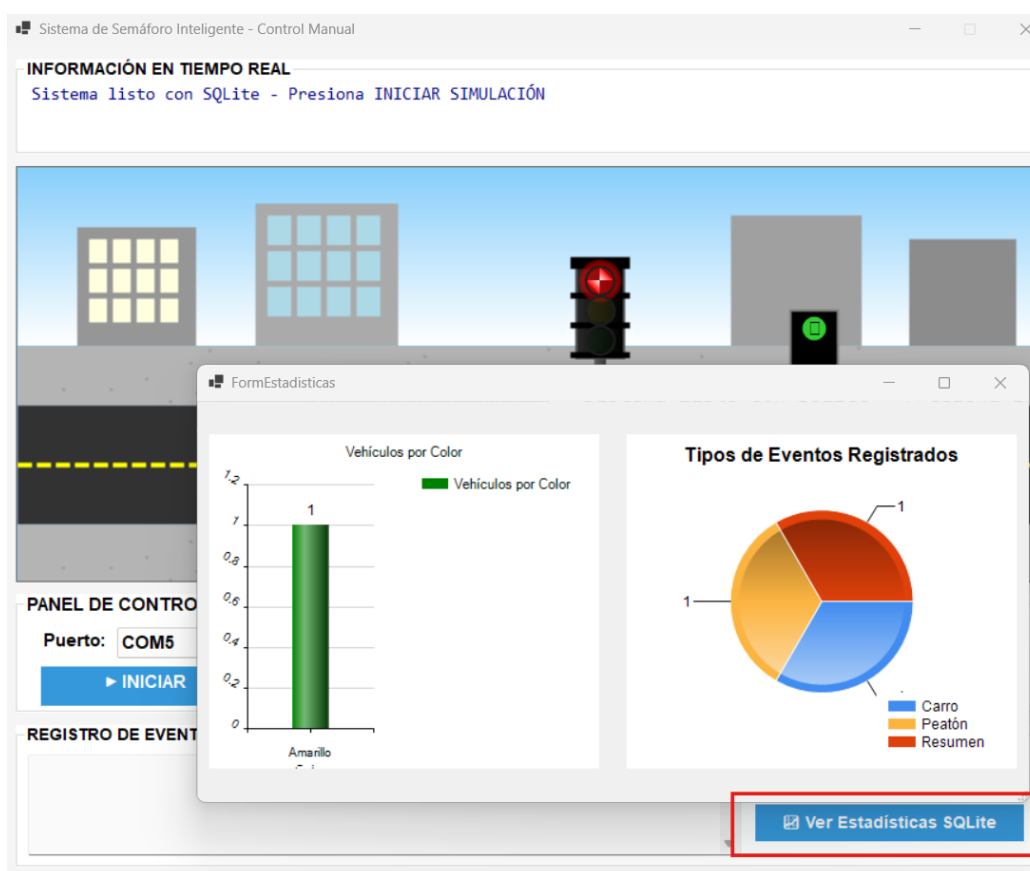


Figura 18: Gráficas generadas en tiempo real desde SQLite.

6.11. Interacción total y validación

El sistema final permite:

- Interactuar desde el pulsador físico o virtual.



- Controlar el semáforo físico o simulado.
- Ver resultados en pantalla, display y LEDs.
- Almacenar y analizar información en base de datos.

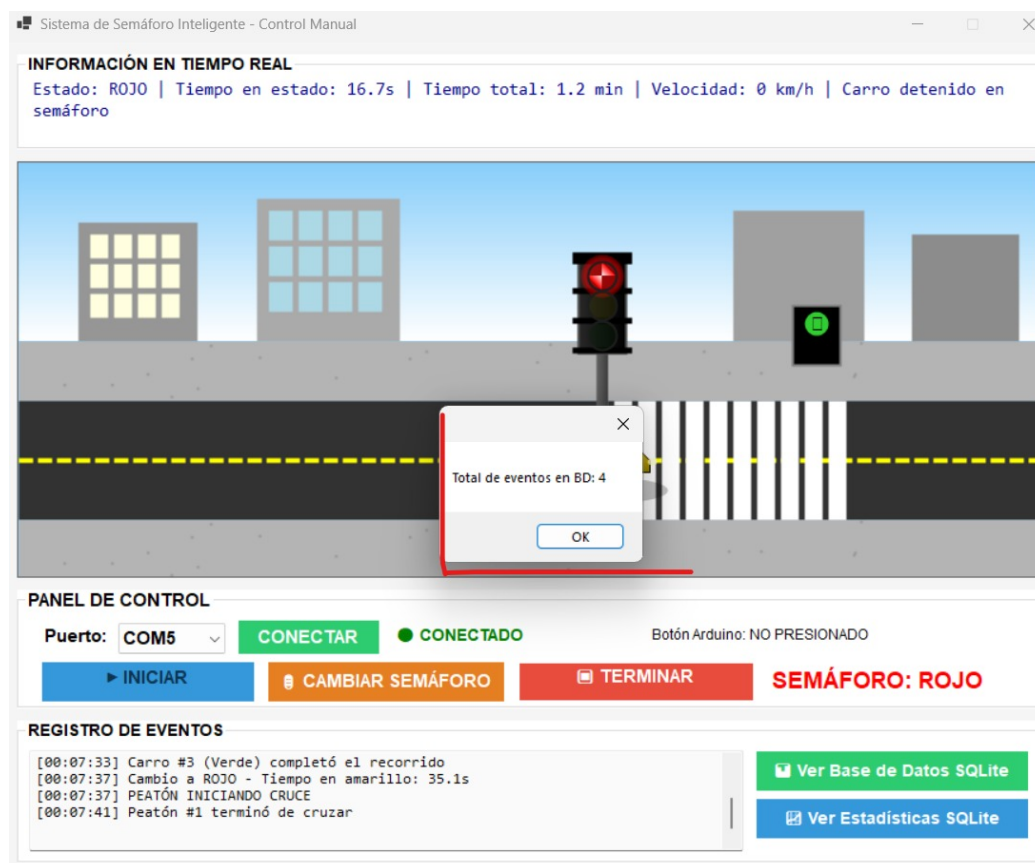


Figura 19: Programa C# corriendo.

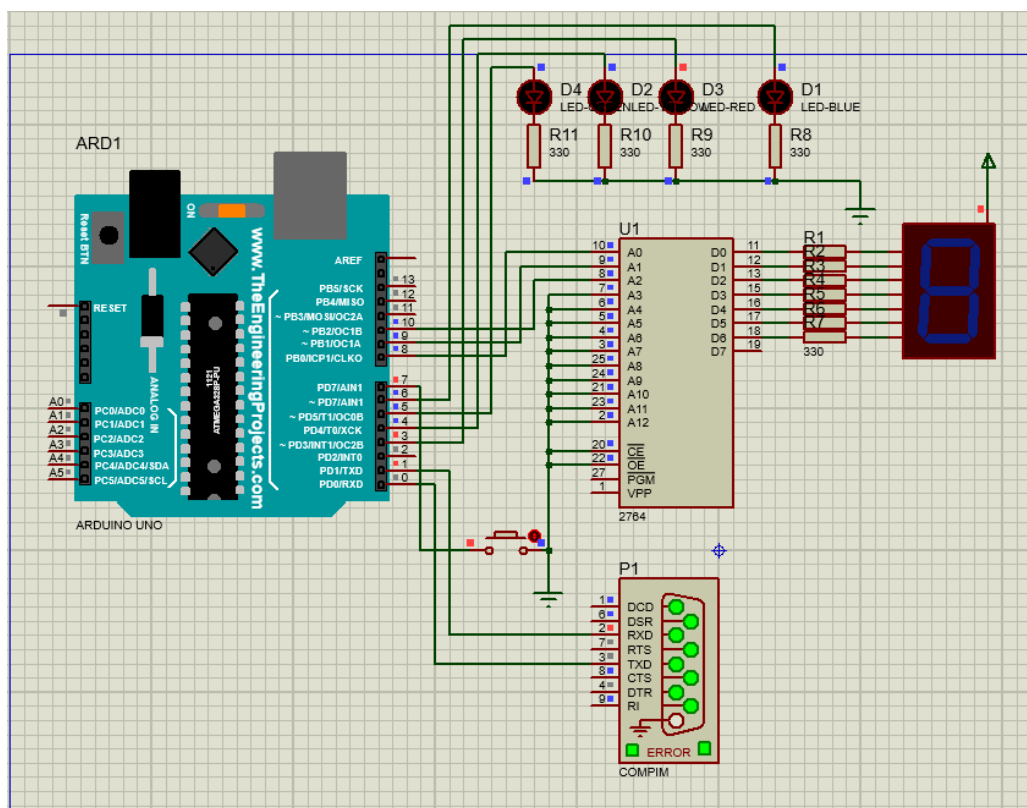


Figura 20: Simulación Proteus 8 corriendo.

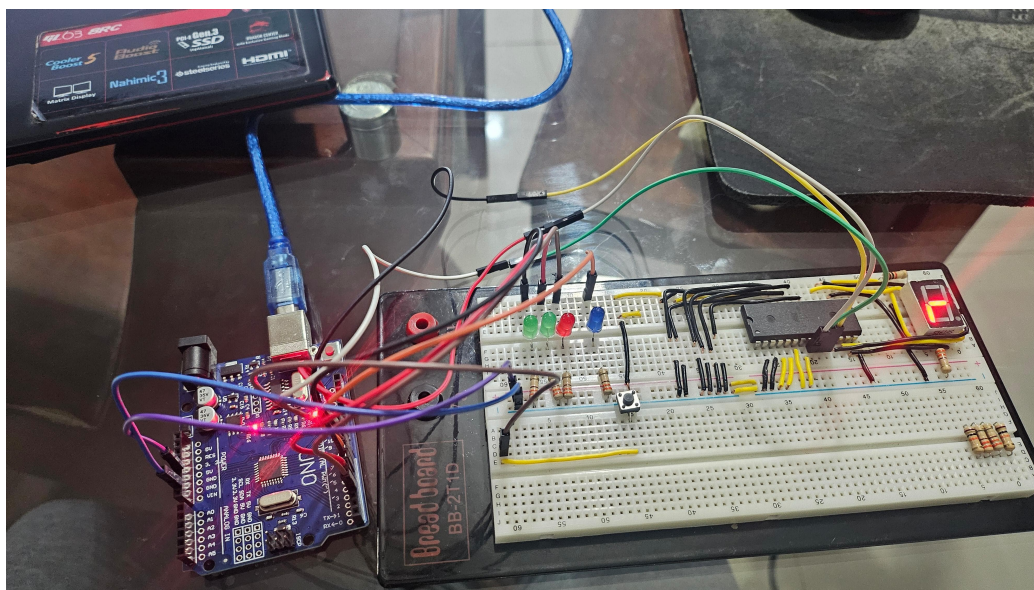


Figura 21: Hardware funcionando y corriendo exitosamente.

La integración completa entre hardware, software y simulación validó el correcto funcionamiento del sistema tanto física como virtualmente.

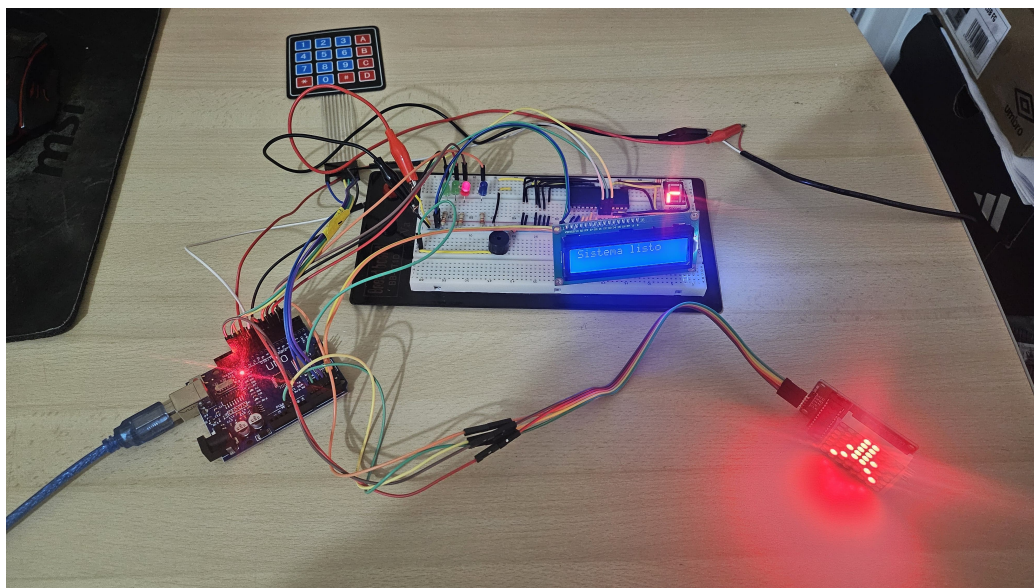


Figura 22: Sistema completo con LCD 16x2, teclado matricial 4x4 y matriz de LEDs funcionando.

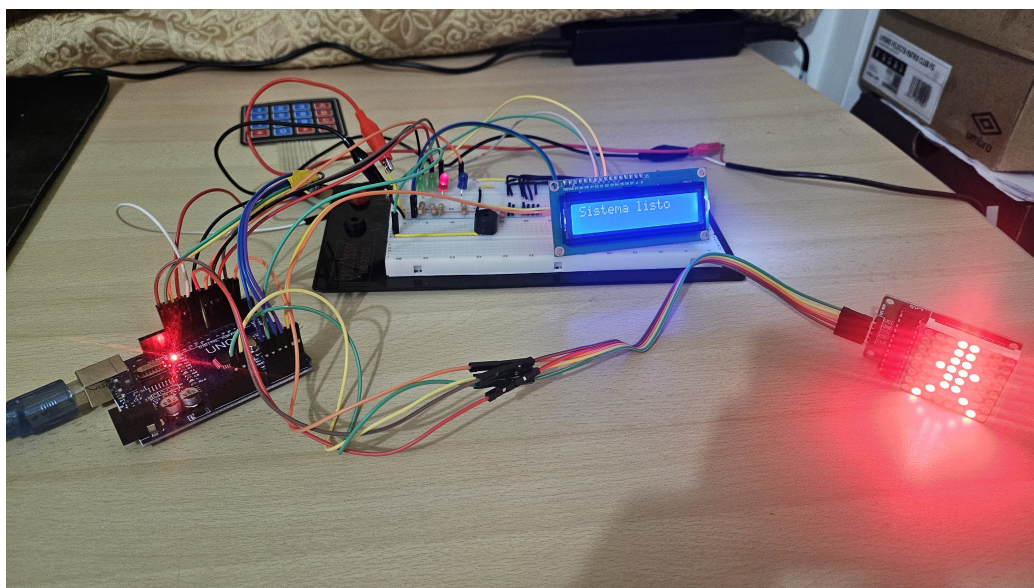


Figura 23: Vista detallada del montaje con todos los componentes integrados.

7. Configuración de Pines y Componentes del Sistema

El proyecto de semáforo inteligente requiere una configuración específica de pines en el microcontrolador Arduino UNO para conectar todos los módulos del sistema. La distribución se diseñó optimizando el uso de los pines disponibles y considerando las características eléctricas de cada componente.

7.1. Distribución Completa de Pines del Arduino UNO

7.2. Diagrama Visual de Conexiones del Sistema

Para facilitar la comprensión del cableado y las interconexiones entre todos los componentes del sistema, se desarrolló un diagrama visual completo que muestra la distribución de pines y las conexiones de cada módulo con el microcontrolador Arduino UNO.

Diagrama de Conexiones - Semáforo Inteligente

Diagrama Visual de Conexiones

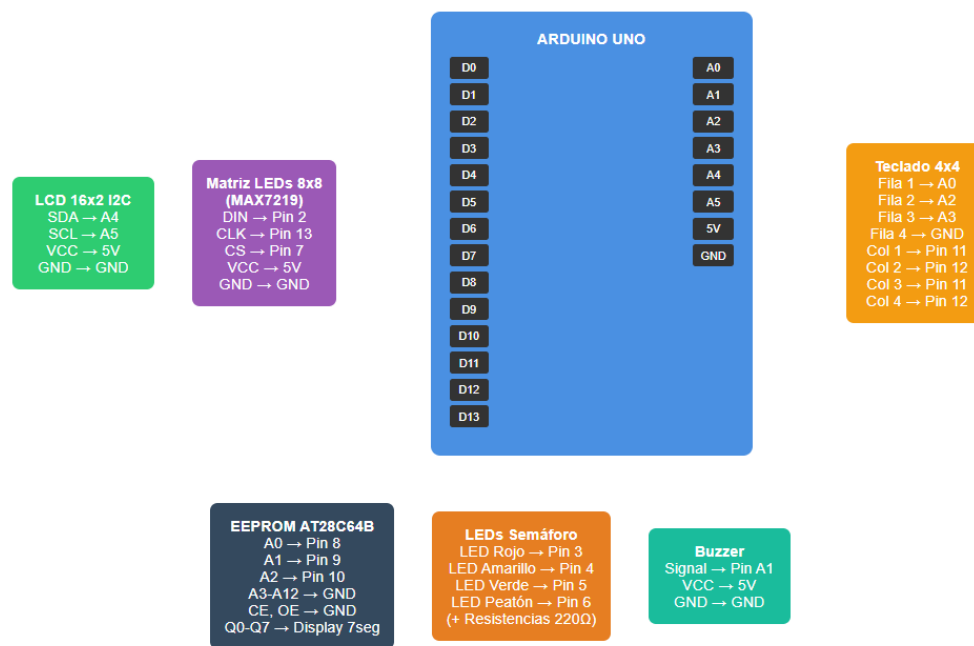


Figura 24: Diagrama visual completo de conexiones del sistema de semáforo inteligente.

El diagrama anterior presenta una vista esquemática de todos los componentes conectados al Arduino UNO, incluyendo:

- **LCD 16x2 I2C:** Conectado a los pines SDA (A4) y SCL (A5) para comunicación I2C
- **Matriz LEDs 8x8 (MAX7219):** Utiliza comunicación SPI con pines DIN (2), CLK (13) y CS (7)
- **Teclado 4x4:** Configurado con 4 filas y 4 columnas distribuidas en los pines analógicos y digitales
- **EEPROM AT28C64B:** Conectada con líneas de dirección A0-A2 y salidas Q0-Q7 hacia el display de 7 segmentos
- **LEDs del semáforo:** Distribuidos en pines digitales con resistencias de protección
- **Buzzer:** Conectado al pin A1 para generación de señales PWM



7.3. Tabla Detallada de Conexiones por Componente

La siguiente tabla proporciona una referencia completa de las conexiones realizadas, especificando la función de cada pin y las observaciones técnicas importantes para cada conexión.

Tabla Detallada de Conexiones

Pin Arduino	Componente	Función	Observaciones
D2	MAX7219	DIN (Datos)	Matriz LEDs 8x8
D3	LED Rojo	Control semáforo	+ Resistencia 220Ω
D4	LED Amarillo	Control semáforo	+ Resistencia 220Ω
D5	LED Verde	Control semáforo	+ Resistencia 220Ω
D6	LED Peatón	Control peatonal	+ Resistencia 220Ω
D7	MAX7219	CS (Chip Select)	Matriz LEDs 8x8
D8	EEPROM	Dirección A0	Display 7 segmentos
D9	EEPROM	Dirección A1	Display 7 segmentos
D10	EEPROM	Dirección A2	Display 7 segmentos
D11	Teclado	Columnas 1,3	Teclas 1,4,7,*,3,6,9,#
D12	Teclado	Columnas 2,4	Teclas 2,5,8,0,A,B,C,D
D13	MAX7219	CLK (Reloj)	Matriz LEDs 8x8
A0	Teclado	Fila 1	Teclas 1,2,3,A
A1	Buzzer	Señal PWM	Melodías del sistema
A2	Teclado	Fila 2	Teclas 4,5,6,B
A3	Teclado	Fila 3	Teclas 7,8,9,C
A4	LCD I2C	SDA (Datos)	Comunicación I2C
A5	LCD I2C	SCL (Reloj)	Comunicación I2C
5V	Todos	Alimentación	+5V sistema
GND	Todos	Tierra común	0V referencia

Figura 25: Tabla detallada de conexiones pin por pin del sistema completo.



7.3.1. Detalles Técnicos de las Conexiones

Pin Arduino	Componente	Función Específica	Observaciones Técnicas
D2	MAX7219	DIN (Data Input)	Entrada de datos seriales para matriz LED
D3	LED Rojo	Control de estado rojo	Resistencia de 220 en serie
D4	LED Amarillo	Control de estado amarillo	Resistencia de 220 en serie
D5	LED Verde	Control de estado verde	Resistencia de 220 en serie
D6	LED Peatón	Control paso peatonal	Resistencia de 220 en serie
D7	MAX7219	CS (Chip Select)	Selección de dispositivo SPI
D8-D10	EEPROM	Direcciones A0-A2	Control de dirección de memoria
D11-D12	Teclado 4x4	Columnas 1 y 3	Configuración matricial
D13	MAX7219	CLK (Clock)	Señal de reloj SPI
A0	Teclado 4x4	Fila 1	Lectura digital de fila
A1	Buzzer	Señal PWM	Generación de tonos y melodías
A2-A3	Teclado 4x4	Filas 2 y 3	Lectura digital de filas
A4-A5	LCD I2C	SDA/SCL	Comunicación I2C estándar

Cuadro 3: Especificaciones técnicas detalladas de cada conexión del sistema.

7.3.2. Consideraciones de Diseño del Cableado

El diseño del cableado se realizó siguiendo las mejores prácticas de sistemas embebidos:

- **Separación de señales:** Las líneas de alimentación se mantuvieron separadas de las líneas de datos para minimizar interferencias
- **Resistencias de protección:** Todos los LEDs incluyen resistencias de 220 para limitar la corriente y proteger tanto los LEDs como los pines del Arduino
- **Pull-up interno:** El teclado matricial utiliza las resistencias pull-up internas del Arduino para estabilizar las lecturas
- **Comunicación I2C:** El LCD utiliza solo 2 pines gracias al módulo I2C, optimizando el uso de pines disponibles
- **Comunicación SPI:** La matriz LED MAX7219 utiliza comunicación SPI eficiente con solo 3 pines de control

7.4. Diagrama ASCII del Sistema Completo

Para complementar el diagrama visual, se incluye una representación ASCII que muestra la arquitectura general del sistema:

Diagrama ASCII Detallado

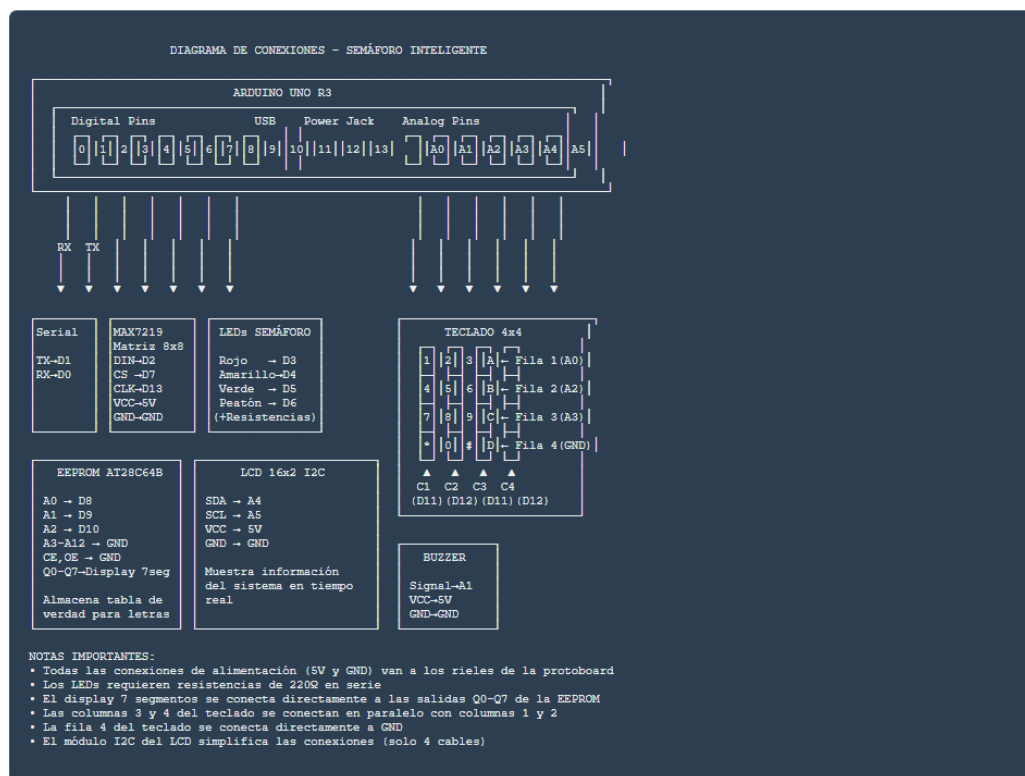


Figura 26: Diagrama ASCII detallado mostrando todas las interconexiones del sistema.

Este diagrama ASCII proporciona una vista técnica detallada que incluye:

- Numeración específica de todos los pines utilizados
- Identificación clara de cada componente y su ubicación
- Especificación de los protocolos de comunicación utilizados
- Notas técnicas importantes sobre alimentación y conexiones críticas
- Referencias a las resistencias y componentes pasivos necesarios



La documentación completa de las conexiones facilita el mantenimiento, la replicación del proyecto y la detección de errores durante el montaje del sistema físico.

Pin Arduino	Uso Actual	Módulo / Descripción
2	DIN de MAX7219	Datos para la matriz de LEDs 8x8
3	LED_ROJO	LED rojo del semáforo
4	LED_AMARILLO	LED amarillo del semáforo
5	LED_VERDE	LED verde del semáforo
6	LED_PEATON	LED peatonal (verde caminante)
7	CS de MAX7219	Chip Select de la matriz de LEDs 8x8
8	EEPROM_A0	Línea de dirección A0 de EEPROM
9	EEPROM_A1	Línea de dirección A1 de EEPROM
10	EEPROM_A2	Línea de dirección A2 de EEPROM
11	Columna 1 teclado 4x4	Parte de teclado matricial
12	Columna 2 teclado 4x4	Parte de teclado matricial
13	CLK de MAX7219	Reloj para matriz de LEDs 8x8
A0	Fila 1 teclado 4x4	Primera fila del teclado matricial
A1	PIN_BUZZER	Buzzer para melodías según estado
A2	Fila 2 teclado 4x4	Segunda fila del teclado matricial
A3	Fila 3 teclado 4x4	Tercera fila del teclado matricial
A4	SDA (I2C)	Comunicación I2C para LCD
A5	SCL (I2C)	Comunicación I2C para LCD

Cuadro 4: Asignación completa de pines del Arduino UNO en el proyecto actualizado.

7.5. Agrupación por Módulos Funcionales

7.5.1. Sistema de Semáforo LED

El control visual principal del semáforo utiliza 4 LEDs independientes:

- **LED_ROJO:** Pin 3 - Señal de alto para vehículos
- **LED_AMARILLO:** Pin 4 - Señal de precaución
- **LED_VERDE:** Pin 5 - Señal de paso libre para vehículos
- **LED_PEATON:** Pin 6 - Señal de cruce peatonal permitido



7.5.2. Display de 7 Segmentos con EEPROM AT28C64B

La visualización de letras correspondientes al color del vehículo utiliza:

- **EEPROM_A0:** Pin 8 - Bit menos significativo de dirección
- **EEPROM_A1:** Pin 9 - Bit medio de dirección
- **EEPROM_A2:** Pin 10 - Bit más significativo de dirección
- **Direcciones A3-A12:** Conectadas a GND (dirección fija)
- **CE y OE:** Conectados a GND (habilitación permanente)
- **Salidas Q0-Q7:** Conectadas directamente a segmentos a-g del display

7.6. Sistema de Entrada con Teclado Matricial 4x4

Se implementó un teclado matricial completo de 4x4 que permite al usuario interactuar directamente con el sistema sin necesidad de la interfaz gráfica, proporcionando 16 funciones diferentes de control.

7.6.1. Configuración del Teclado

1	2	3	A
4	5	6	B
7	8	9	C
*	0	#	D

Cuadro 5: Distribución física del teclado matricial 4x4.

7.6.2. Funciones Asignadas a cada Tecla

- **Tecla "1":** Iniciar simulación automática
- **Tecla "2":** Cambiar estado del semáforo manualmente
- **Tecla "3":** Terminar simulación y guardar datos
- **Tecla ".^":** Mostrar estadísticas en pantalla
- **Tecla "4":** Ajustar tiempo de estado verde (+5 segundos)



- Tecla "5": Ajustar tiempo de estado amarillo (+2 segundos)
- Tecla "6": Ajustar tiempo de estado rojo (+5 segundos)
- Tecla "B": Activar modo nocturno (tiempos reducidos)
- Tecla "7": Mostrar información del sistema en LCD
- Tecla "8": Configurar brillo de matriz LED
- Tecla "9": Ejecutar test de todos los componentes
- Tecla "C": Acceder al modo de configuración avanzada
- Tecla "*": Navegación hacia atrás en menús
- Tecla "0": Confirmar selección en menús
- Tecla "#": Navegación hacia adelante en menús
- Tecla "D": Guardar configuración actual en EEPROM

7.6.3. Implementación en el Código C#

```
1 private void Arduino_DataReceived(object sender,
   SerialDataReceivedEventArgs e)
2 {
3     string datos = arduino.ReadLine().Trim();
4
5     if (datos.StartsWith("Tecla presionada: "))
6     {
7         string tecla = datos.Substring(18);
8
9         switch (tecla)
10        {
11            case "1":
12                BtnIniciar_Click(null, null);
13                ReproducirSonido("start.wav");
14                break;
15            case "2":
16                BtnCambiar_Click(null, null);
```



```
17         ReproducirSonido("clic.wav");
18         break;
19     case "A":
20         BtnEstadisticas_Click(null, null);
21         ReproducirSonido("data.wav");
22         break;
23     }
24 }
25 }
```

Listing 3: Manejo de teclas del teclado matricial

7.6.4. Matriz de LEDs 8x8 (MAX7219)

La visualización gráfica avanzada utiliza comunicación SPI:

- **DIN:** Pin 2 - Línea de datos seriales
- **CLK:** Pin 13 - Señal de reloj
- **CS:** Pin 7 - Chip Select para selección del dispositivo

7.7. Sistema de Audio Avanzado con Buzzer

El buzzer integrado reproduce melodías específicas que corresponden a cada estado del semáforo, proporcionando retroalimentación auditiva para personas con discapacidad visual y mejorando la experiencia del usuario.

7.7.1. Melodías Programadas por Estado

- **Verde:** Melodía ascendente de 3 tonos (indicando .avanzar")
- **Amarillo:** Dos tonos alternados (indicando "precaución")
- **Rojo:** Tono grave prolongado (indicando .alto")
- **Peatonal:** Secuencia rítmica (indicando cruzar")



7.7.2. Implementación de Melodías

Listing 4: Generación de melodías según estado del semáforo

```
#define PIN_BUZZER A1

// Frecuencias musicales
#define DO 261
#define RE 294
#define MI 329
#define SOL 392

void melodiaVerde() {
    tone(PIN_BUZZER, DO, 200);
    delay(250);
    tone(PIN_BUZZER, MI, 200);
    delay(250);
    tone(PIN_BUZZER, SOL, 300);
    delay(350);
}

void melodiaRojo() {
    tone(PIN_BUZZER, 150, 800); // Tono grave prolongado
    delay(1000);
}
```

7.7.3. Integración con Estados del Sistema

El sistema de audio se sincroniza automáticamente con los cambios de estado del semáforo, proporcionando feedback auditivo inmediato tanto para el hardware físico como para la simulación en Proteus.



7.8. Display LCD 16x2 con Interfaz I2C

El sistema integra un display LCD de 16 caracteres por 2 líneas con módulo I2C para mostrar información del estado del sistema en tiempo real, reduciendo el número de pines necesarios y simplificando las conexiones.

7.8.1. Características del Módulo LCD I2C

- Display HD44780 de 16x2 caracteres
- Módulo PCF8574 para comunicación I2C
- Dirección I2C configurable (por defecto 0x27)
- Retroiluminación LED azul
- Alimentación de 5V desde Arduino

7.8.2. Información Mostrada en el LCD

El LCD presenta diferentes tipos de información según el contexto:

- **Línea 1:** Estado actual del semáforo y tiempo restante
- **Línea 2:** Información de vehículos y estadísticas

Estado del Sistema	Información en LCD
Inicio	"Sistema Listo/ "Presiona tecla 1"
Verde	"VERDE: 15s/ Carros: 5 V:3 R:2"
Amarillo	".AMARILLO: 3s/ Reducir velocidad"
Rojo	ROJO: 10s/ "Peaton cruzando"
Estadísticas	"Total: 25 carros/ "Peatones: 8"

Cuadro 6: Mensajes mostrados en el LCD según el estado del sistema.

7.8.3. Conexión I2C

- **SDA:** Pin A4 del Arduino UNO
- **SCL:** Pin A5 del Arduino UNO

- **VCC:** 5V del Arduino
- **GND:** Tierra común

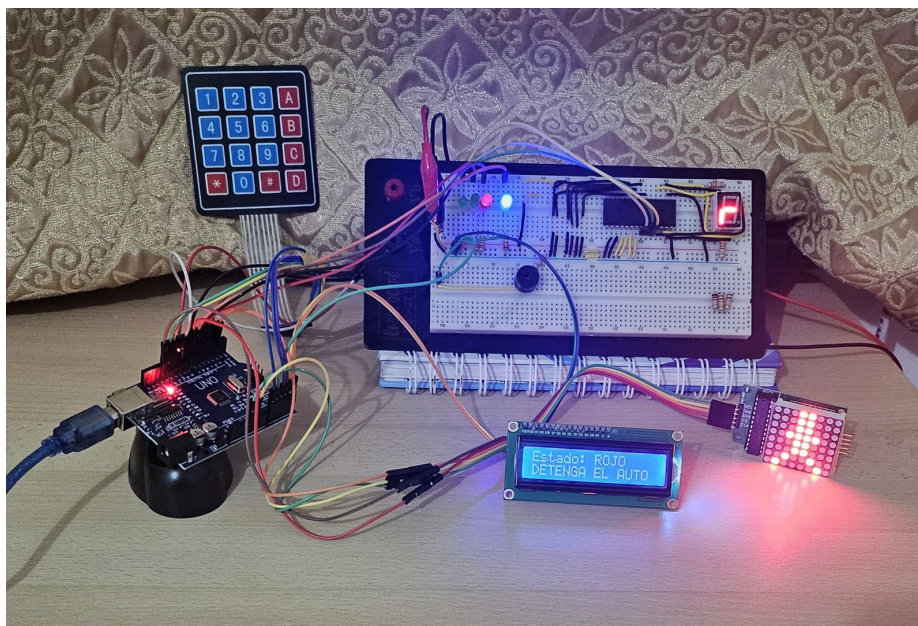


Figura 27: Display LCD 16x2 mostrando información del sistema en tiempo real.

“

El LCD muestra información en tiempo real del estado del sistema, incluyendo:

- Estado actual del semáforo (Verde, Amarillo, Rojo, Peatonal)
- Tiempo restante para cambio de estado
- Mensajes de configuración del sistema
- Información de vehículos detectados

7.9. Sistema de Matriz de LEDs 8x8 con MAX7219

El sistema incorpora una matriz de LEDs 8x8 controlada por el integrado MAX7219, que permite mostrar patrones gráficos y animaciones relacionadas con el estado actual del semáforo. Esta funcionalidad añade valor visual y educativo al proyecto.



7.9.1. Características del MAX7219

- Controlador serial para displays de 7 segmentos y matrices de LEDs
- Interfaz SPI de 3 cables (DIN, CLK, CS)
- Control de intensidad programable
- Multiplexación automática sin parpadeo
- Capacidad de cascada para múltiples displays

7.9.2. Patrones Implementados

Se programaron diferentes patrones para cada estado del semáforo:

- **Estado Verde:** Flecha hacia adelante animada
- **Estado Amarillo:** Símbolo de precaución parpadeante
- **Estado Rojo:** Símbolo de alto estático
- **Estado Peatonal:** Figura de persona caminando

7.9.3. Animaciones Implementadas

Se desarrollaron secuencias animadas de 8 frames para enriquecer la experiencia visual:

- **Animación de Flecha:** Movimiento de izquierda a derecha indicando dirección del tráfico
- **Parpadeo de Advertencia:** Secuencia intermitente durante estado amarillo
- **Símbolo de Alto Pulsante:** Variación de intensidad durante estado rojo
- **Figura Peatonal Caminando:** Animación de persona en movimiento durante cruce peatonal

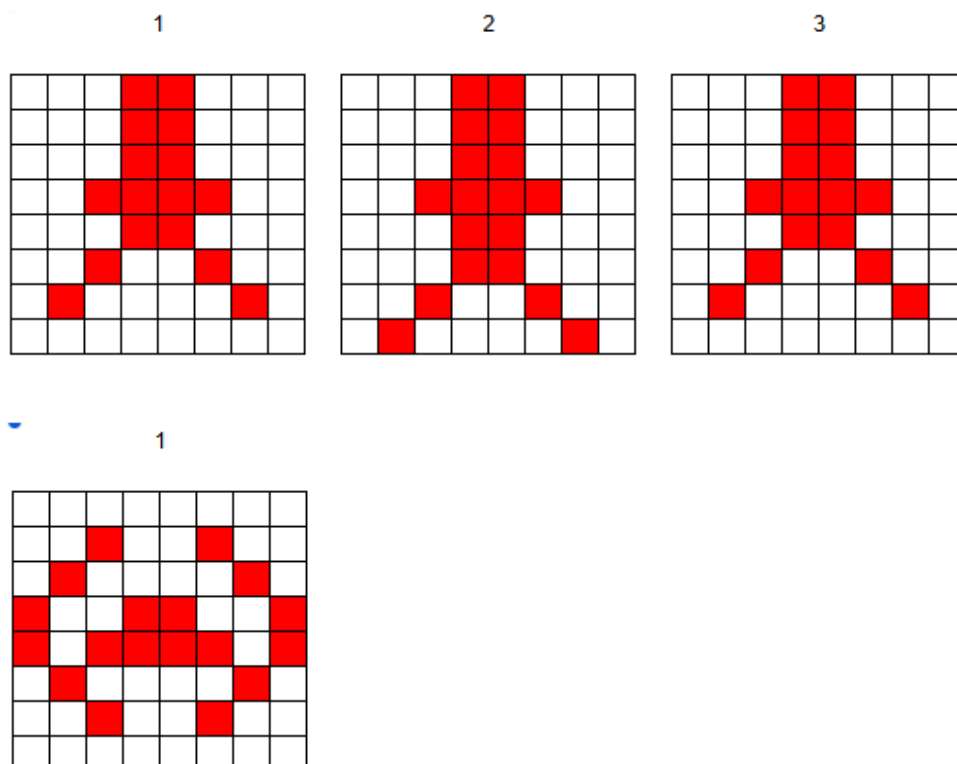


Figura 28: Secuencia de 8 frames de la animación de flecha en matriz de LEDs 8x8.

Cada frame se actualiza cada 200ms, creando una animación fluida que mejora la percepción visual del estado del semáforo.

7.9.4. Código de Control de la Matriz

Listing 5: Control de matriz LED con MAX7219

```
#include <LedControl.h>

LedControl lc = LedControl(2, 13, 7, 1); // DIN=2, CLK=13, CS=7

// Patrones para cada estado (8x8 bits)
byte patron_flecha[8] = {
    0b00001000,
    0b00011100,
```



```
0b00111110 ,
0b01111111 ,
0b00111110 ,
0b00011100 ,
0b00001000 ,
0b00000000
};

void mostrarPatron(byte* patron) {
    for (int fila = 0; fila < 8; fila++) {
        lc.setRow(0, fila , patron[fila]);
    }
}
```

8. Implementación de Funcionalidades Avanzadas

8.1. Sistema de Matriz de LEDs 8x8 con MAX7219

El sistema incorpora una matriz de LEDs 8x8 controlada por el integrado MAX7219, que permite mostrar patrones gráficos y animaciones relacionadas con el estado actual del semáforo. Esta funcionalidad añade valor visual y educativo al proyecto.

8.1.1. Características del MAX7219

- Controlador serial para displays de 7 segmentos y matrices de LEDs
- Interfaz SPI de 3 cables (DIN, CLK, CS)
- Control de intensidad programable
- Multiplexación automática sin parpadeo
- Capacidad de cascada para múltiples displays

8.1.2. Patrones Implementados

Se programaron diferentes patrones para cada estado del semáforo:



- **Estado Verde:** Flecha hacia adelante animada
- **Estado Amarillo:** Símbolo de precaución parpadeante
- **Estado Rojo:** Símbolo de alto estático
- **Estado Peatonal:** Figura de persona caminando

8.1.3. Animaciones Implementadas

Se desarrollaron secuencias animadas de 8 frames para enriquecer la experiencia visual:

- **Animación de Flecha:** Movimiento de izquierda a derecha indicando dirección del tráfico
- **Parpadeo de Advertencia:** Secuencia intermitente durante estado amarillo
- **Símbolo de Alto Pulsante:** Variación de intensidad durante estado rojo
- **Figura Peatonal Caminando:** Animación de persona en movimiento durante cruce peatonal

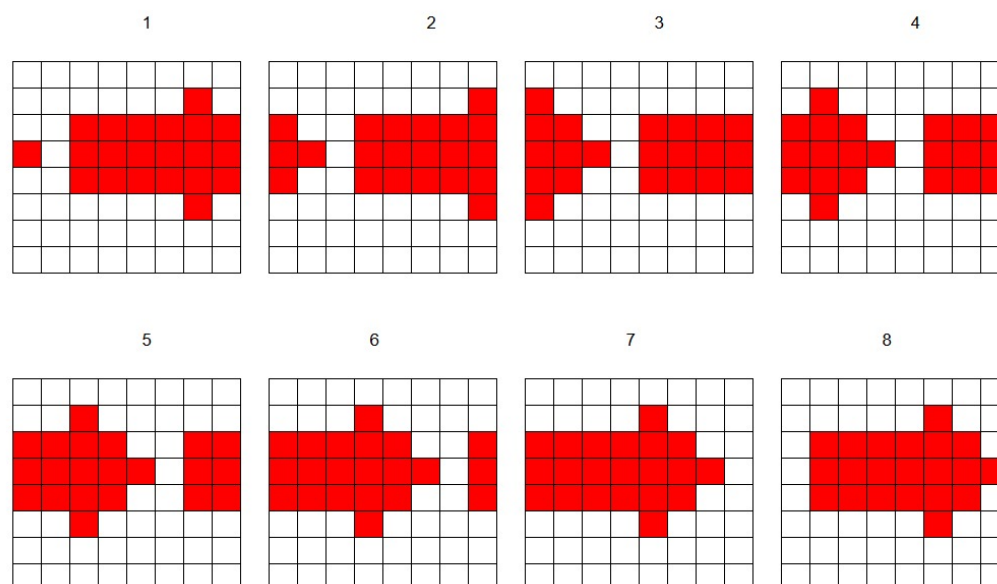


Figura 29: Secuencia de 8 frames de la animación de flecha en matriz de LEDs 8x8.



Cada frame se actualiza cada 200ms, creando una animación fluida que mejora la percepción visual del estado del semáforo.

8.2. Sistema de Audio con Buzzer

El buzzer integrado reproduce melodías específicas que corresponden a cada estado del semáforo, proporcionando retroalimentación auditiva para personas con discapacidad visual y mejorando la experiencia del usuario.

8.2.1. Melodías Programadas

- **Verde:** Melodía ascendente de 3 tonos (indicando "avanzar")
- **Amarillo:** Dos tonos alternados (indicando "precaución")
- **Rojo:** Tono grave prolongado (indicando "alto")
- **Peatonal:** Secuencia rítmica (indicando "cruzar")

8.3. Teclado de Control Simplificado

Se implementó un teclado matricial en configuración 1x4 que permite al usuario interactuar directamente con el sistema sin necesidad de la interfaz gráfica.

8.3.1. Funciones de las Teclas

- **Tecla "1":** Activar modo manual del semáforo
- **Tecla "2":** Activar modo automático con temporizadores
- **Tecla "3":** Solicitar paso peatonal inmediato
- **Tecla "4":** Ajustar tiempo verde (+5 segundos)
- **Tecla "5":** Ajustar tiempo amarillo (+2 segundos)
- **Tecla "6":** Ajustar tiempo rojo (+5 segundos)
- **Tecla "7":** Mostrar estadísticas en LCD
- **Tecla "8":** Configurar brillo de matriz LED



- **Tecla "9":** Test de todos los componentes
- **Tecla ".A":** Resetear sistema y volver al estado inicial
- **Tecla "B":** Modo nocturno (tiempos reducidos)
- **Tecla Ç":** Modo configuración avanzada
- **Tecla "D":** Guardar configuración en EEPROM
- **Teclas "0", "*", "#":** Navegación en menús LCD

9. Integración y Comunicación entre Módulos

9.1. Protocolo de Comunicación Serial

El sistema utiliza comunicación serial UART a 9600 baudios para la sincronización entre la interfaz gráfica en Visual Studio y el microcontrolador Arduino. Los mensajes siguen un formato estructurado para garantizar la integridad de los datos.

9.1.1. Formato de Mensajes

- **Comando de estado:** STATE:X donde X es el código del estado (0-7)
- **Evento de vehículo:** VEHICLE:COLOR:TIME
- **Evento peatonal:** PEDESTRIAN:TIME
- **Confirmación:** ACK:COMMAND

9.2. Sincronización Hardware-Software

La sincronización entre el prototipo físico y la simulación en Proteus se logra mediante:

- Uso de puertos seriales virtuales para la comunicación con Proteus
- Protocolo común de mensajes entre ambos entornos
- Validación cruzada de estados entre hardware y software
- Manejo de timeouts para detectar desconexiones

10. Resultados y Validación del Sistema

10.1. Pruebas de Funcionalidad

Se realizaron pruebas exhaustivas para validar cada componente y la integración completa del sistema. A continuación se muestran evidencias fotográficas del hardware funcionando correctamente en condiciones reales de operación.

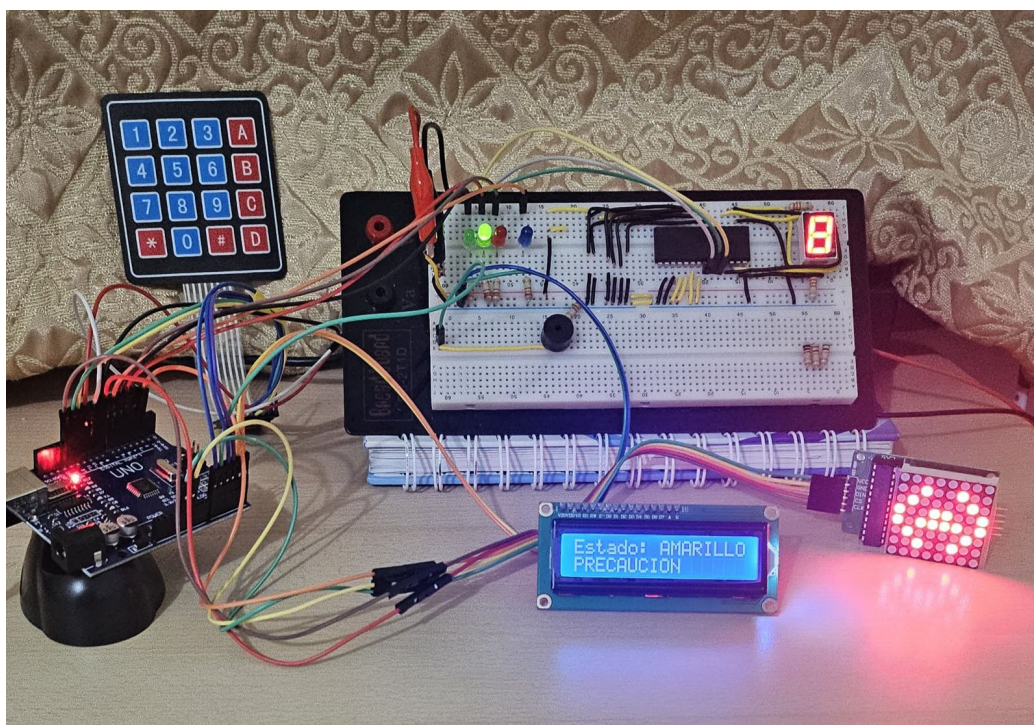


Figura 30: Sistema completo en funcionamiento mostrando todos los componentes operativos: LEDs del semáforo, display 7 segmentos, matriz LED 8x8, LCD 16x2 y teclado matricial.

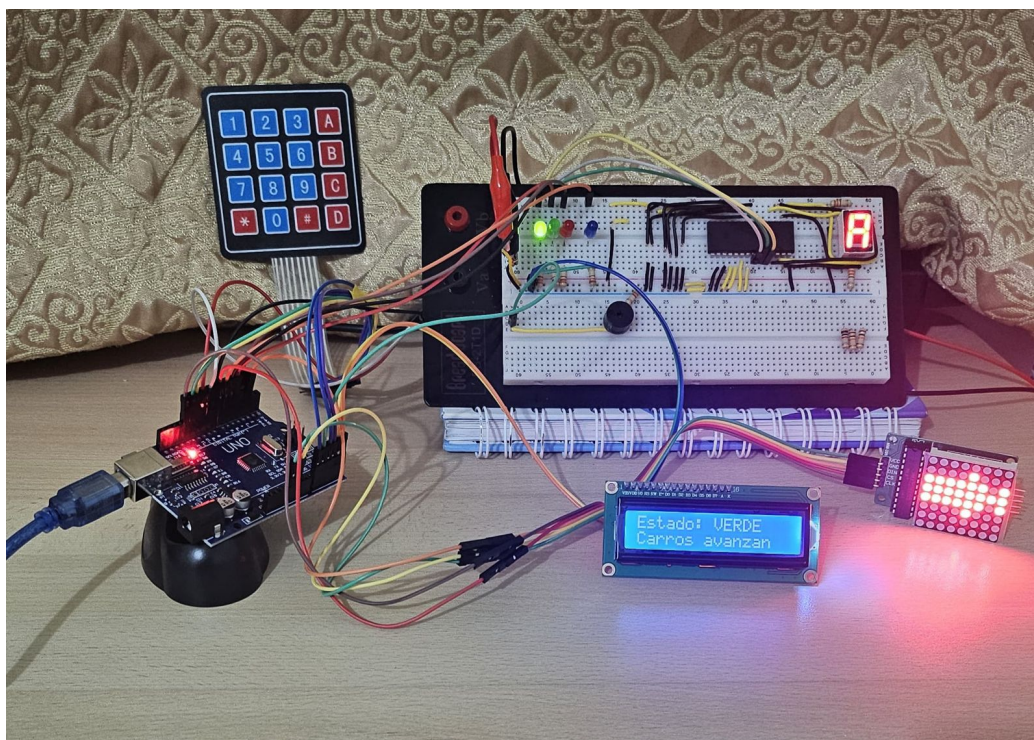


Figura 31: Vista detallada del sistema durante una prueba de funcionamiento, evidenciando la correcta visualización en todos los displays y la sincronización entre componentes.

Las pruebas realizadas validaron exitosamente el funcionamiento integral del sistema, demostrando la correcta implementación de todas las funcionalidades diseñadas.

10.1.1. Pruebas de Hardware

- Verificación de la lectura correcta de la EEPROM en todas las direcciones
- Validación del display de 7 segmentos mostrando las letras correctas
- Prueba de todos los LEDs del semáforo en secuencia
- Verificación del funcionamiento del teclado matricial
- Validación de la matriz de LEDs 8x8 con diferentes patrones
- Prueba del buzzer con todas las melodías programadas



10.1.2. Pruebas de Software

- Validación de la comunicación serial en ambos modos (físico y virtual)
- Prueba de inserción y consulta de datos en SQLite
- Verificación de la generación correcta de gráficos estadísticos
- Validación de la interfaz gráfica en diferentes resoluciones
- Prueba de manejo de errores y excepciones

10.1.3. Pruebas de Integración

- Sincronización correcta entre hardware físico y interfaz gráfica
- Funcionamiento simultáneo de Proteus y Visual Studio
- Validación de la coherencia de datos entre todos los módulos
- Prueba de conmutación entre modo físico y virtual sin pérdida de datos

10.2. Métricas de Rendimiento

Métrica	Valor Medido	Objetivo
Tiempo de respuesta del sistema	<100ms	<200ms
Tiempo de cambio de estado	250ms	<500ms
Precisión de temporización	$\pm 2\%$	$\pm 5\%$
Tasa de errores de comunicación	0.1 %	<1 %
Capacidad de almacenamiento (eventos/hora)	3600	>1000

Cuadro 7: Métricas de rendimiento del sistema completo.

10.3. Casos de Uso Validados

1. **Operación normal automática:** El semáforo funciona con temporizadores predefinidos
2. **Solicitud peatonal:** El sistema responde a solicitudes de paso peatonal
3. **Modo manual:** Control directo desde el teclado o interfaz gráfica



4. **Registro de eventos:** Todos los eventos se almacenan correctamente
5. **Visualización estadística:** Los gráficos reflejan accuradamente los datos almacenados
6. **Operación en modo simulado:** Proteus replica fielmente el comportamiento físico

11. Sistema de Retroalimentación Sonora

El proyecto incorpora un sistema completo de retroalimentación sonora que mejora la experiencia del usuario y proporciona accesibilidad para personas con discapacidad visual.

11.1. Archivos de Sonido Implementados

El sistema utiliza archivos WAV que se reproducen desde la aplicación C# para diferentes eventos:

- **start.wav:** Sonido al iniciar la simulación
- **clic.wav:** Sonido al cambiar estado del semáforo
- **close.wav:** Sonido al terminar la simulación
- **data.wav:** Sonido al abrir ventana de estadísticas

11.2. Implementación de Reproducción de Audio

```
1 private void ReproducirSonido(string nombreArchivo)
2 {
3     try
4     {
5         string ruta = Path.Combine(Application.StartupPath, "Sonidos", nombreArchivo);
6         if (File.Exists(ruta))
7         {
8             SoundPlayer player = new SoundPlayer(ruta);
9             player.Play();
10        }
11    }
```



```
12     catch (Exception ex)
13     {
14         Console.WriteLine("Error al reproducir sonido: " + ex.
15             Message);
16     }
```

Listing 6: Método para reproducir sonidos en C#

11.3. Integración con Hardware

El sistema combina los sonidos del software (archivos WAV) con las melodías del buzzer físico, creando una experiencia auditiva completa que incluye:

- Sonidos de la interfaz gráfica (archivos WAV)
- Melodías del buzzer según estado del semáforo
- Feedback auditivo al presionar teclas del teclado matricial
- Señales sonoras para personas con discapacidad visual

12. Conclusiones

- La implementación de la matriz de LEDs 8x8 con MAX7219 demostró ser efectiva para proporcionar retroalimentación visual avanzada, generando animaciones fluidas que mejoran significativamente la experiencia del usuario y facilitan la comprensión del estado actual del sistema.
- El sistema de audio completo, que combina melodías del buzzer físico con archivos WAV en la aplicación, crea una experiencia multimedia que no solo mejora la usabilidad sino que también proporciona accesibilidad para usuarios con discapacidades visuales.
- La integración del teclado matricial 4x4 permitió crear un sistema de control independiente de la interfaz gráfica, demostrando la versatilidad del diseño y la capacidad de operación autónoma del sistema embebido.



- El display LCD 16x2 con interfaz I2C optimizó el uso de pines del microcontrolador mientras proporcionaba información clara y en tiempo real del estado del sistema, validando la importancia de la retroalimentación visual directa en sistemas embebidos.
- La sincronización exitosa entre todos los módulos (matriz LED, buzzer, LCD, teclado, LEDs tradicionales y display 7 segmentos) demuestra la complejidad alcanzada en el proyecto y la efectividad de la programación modular en sistemas embebidos.
- El sistema completo representa una solución integral que abarca desde la lógica de control básica hasta interfaces avanzadas de usuario, estableciendo una base sólida para futuros desarrollos en sistemas de control de tráfico inteligente y automatización urbana.

13. Aplicación a la vida real — Propuesta

Problema identificado

En muchas ciudades, las intersecciones presentan congestión en horas pico, demoras excesivas para peatones, y falta de prioridad para vehículos de emergencia o transporte estratégico. Los semáforos convencionales operan con ciclos fijos que no se adaptan al flujo real, generando:

- Colas largas y tiempos de espera innecesarios.
- Riesgo elevado en cruces peatonales sin asistencia dinámica.
- Dificultad para coordinar convoyes, transporte público y emergencias.
- Uso ineficiente de energía y recursos de mantenimiento.

Solución propuesta

El sistema de semáforo inteligente se adapta al flujo real mediante sensado, control local y comunicación remota, con funciones como:

- Ajuste automático de tiempos según demanda vehicular y peatonal.
- Activación de fases peatonales bajo demanda con aviso sonoro/visual.



- Priorización temporal de vehículos de emergencia y convoyes.
- Modos especiales para lluvia, niebla, noche o eventos masivos.
- Coordinación entre intersecciones para “onda verde” en corredores clave.
- Monitoreo y telemetría para gestión y mantenimiento predictivo.

Dónde aplica (casos de uso concretos)

1. Intersecciones urbanas con congestión recurrente (picos laborales/escolares).
2. Zonas escolares y hospitales (ventanas horarias con refuerzo peatonal).
3. Corredores logísticos y avenidas de alta demanda (“onda verde” sincronizada).
4. Accesos a aeropuertos, parques industriales y terminales terrestres.
5. Cruces peatonales de alto riesgo y pasos de cebra sin semaforizar (modo demanda).
6. Túneles y pasos a desnivel (control por incidentes y ventilación/visibilidad).
7. Áreas turísticas y centros históricos con alta afluencia peatonal.
8. Cruces ferroviarios en superficie (bloqueos coordinados por aproximación).
9. Ciclovías y zonas 30 (fases dedicadas y señalización sonora).
10. Obras viales temporales (fases reversibles, carril alternado).
11. Comunidades y ejes rurales con picos estacionales (mercados, festividades).

Qué se instala

- Sensores de demanda: cámara IP con WDR, radar de velocidad/flujo, lazo inductivo o sensor magnético de piso, botonera peatonal accesible, sensor de luz/lluvia.
- Control local: SBC (cómputo en el borde) y controlador de tráfico con salidas a relés/MOSFET, RTC y registro de eventos.
- Comunicaciones: 4G/LTE o radio; operación autónoma si no hay red; actualización remota segura.



- Energía y protección: luminarias LED, UPS 1–4 h, gabinete IP65/NEMA con protecciones.
- Monitoreo: tablero web con KPIs y exportación de datos.

Cómo se implementa (plan corto)

1. **4 semanas:** aforos y diseño de fases; ubicación de sensores y gabinete.
2. **4–6 semanas:** instalación, puesta a tierra, comunicaciones y pruebas en sitio.
3. **2 semanas:** calibración fina, tablero de KPIs y entrega operativa con capacitación.

Qué medimos (KPIs de éxito)

- % de reducción en tiempo de espera y longitud de cola en hora pico.
- Tiempo de atención a prioridad (emergencia/convoy) y tasa de cumplimiento.
- Seguridad peatonal: incidentes reportados y cumplimiento de cruce seguro.
- Disponibilidad del sistema y tasa de fallas de sensores/luminarias.

Riesgos y mitigaciones

- **Falla de red:** modo local autónomo con plan de ciclos base.
- **Ciberseguridad:** VPN, credenciales rotadas, *firewall* y auditoría de eventos.
- **Ambiente agresivo:** gabinete IP65, pararrayos, supresores y rutina de mantenimiento.



14. Componentes para implementación a escala real

Lista de componentes

Nro	Componente
1	Controlador de tráfico programable
2	Unidad de cómputo en el borde (SBC)
3	Cámara IP PoE con WDR/IR
4	Sensor radar doppler vehicular
5	Lazo inductivo o sensor magnético de piso
6	Botonera peatonal accesible
7	Módulo de audio peatonal
8	Cabezal semafórico LED vehicular 200 mm
9	Cabezal semafórico LED peatonal 200 mm
10	Módulos de potencia (relés de estado sólido / MOSFET)
11	Gabinete exterior IP65/NEMA
12	UPS 1000 VA con baterías
13	Protecciones eléctricas (supresor, breaker, tierra)
14	Módem 4G/LTE industrial o radio
15	Switch PoE industrial y cableado UTP exterior
16	Postes y soportes para cabezales/cámaras
17	Sensores ambientales (pluviómetro, fotocelda, luxómetro)
18	Señalización vertical y marcas viales
19	Multímetro digital
20	Software de tablero y gestión remota

Cuadro 8: Lista de componentes necesarios para la implementación real del sistema.

Función de cada componente

- **Controlador de tráfico programable:** ejecuta la lógica de control de fases y priorización.



- **Unidad de cómputo en el borde (SBC):** procesa datos de sensores y aplica algoritmos adaptativos.
- **Cámara IP PoE con WDR/IR:** detecta flujo, cuenta vehículos y estima colas.
- **Sensor radar doppler:** mide la velocidad y flujo de vehículos.
- **Lazo inductivo o sensor magnético:** detecta presencia de vehículos en carriles.
- **Botonera peatonal accesible:** solicita cruce y da retroalimentación sonora/vibratoria.
- **Módulo de audio peatonal:** emite avisos sonoros para cruce seguro.
- **Cabecal semafórico LED vehicular:** señaliza alto, precaución y paso a vehículos.
- **Cabecal semafórico LED peatonal:** indica al peatón cuándo cruzar o detenerse.
- **Módulos de potencia:** conmutan la energía a luminarias y dispositivos de forma segura.
- **Gabinete exterior IP65/NEMA:** protege los equipos de clima y vandalismo.
- **UPS 1000 VA con baterías:** mantiene el sistema en funcionamiento en cortes eléctricos.
- **Protecciones eléctricas:** previenen daños por sobrecargas o descargas.
- **Módem 4G/LTE industrial o radio:** envía datos y permite control remoto.
- **Switch PoE y cableado:** alimenta y comunica cámaras y sensores.
- **Postes y soportes:** aseguran visibilidad y correcta instalación de dispositivos.
- **Sensores ambientales:** ajustan el sistema a condiciones de lluvia, luz o niebla.
- **Señalización y marcas viales:** refuerzan el mensaje visual del semáforo.
- **Multímetro digital:** permite medir y diagnosticar parámetros eléctricos.
- **Software de tablero y gestión:** supervisa el sistema y genera reportes.